

C Code Lab - Published Questions

Total Questions: 220

1. Phase 1 — Foundation — Introduction to C — Q1: Print Hello World

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Print Hello World. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print exactly: Hello World

Constraints: This program runs without waiting for typed input.

Sample Input:

Expected Output: Hello World

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

Hello World

2. Phase 1 — Foundation — Introduction to C — Q2: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

3. Phase 1 — Foundation — Introduction to C — Q3: Access Array Using Pointer

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Access Array Using Pointer. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of array elements using pointer traversal.

Constraints: Length plus list of ints.

Sample Input:

4

2 4 6 8

Expected Output: 20

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

9

Expected output:

9

Case 2 (public)

Input:

5

1 2 3 4 5

Expected output:

15

Case 3 (public)

Input:

2

-1 4

Expected output:

3

4. Phase 1 — Foundation — Introduction to C — Q4: Print Name And City

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Print Name And City. Read input from stdin and print exact output.

Input Format: Line 1: name. Line 2: city.

Output Format: Print name on line 1, city on line 2.

Constraints: You get two answers the user typed: first the name word, second the city word — each alone on its line.

Sample Input:

Riya

Chennai

Expected Output (sample):

Riya

Chennai

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Ada

Boston

Expected output:

Ada

Boston

Case 2 (public)

Input:

Riya

Chennai

Expected output:

Riya
Chennai

Case 3 (public)**Input:**

Zoe
Paris

Expected output:

Zoe
Paris

5. Phase 1 — Foundation — Introduction to C — Q5: Armstrong Number Check

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Armstrong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Let k be the number of decimal digits of n. n is an Armstrong (narcissistic) number if the sum of (each digit raised to the k-th power) equals n. Print Yes or No. Single-digit n always satisfies this (k=1), so every 1–9 is Yes.

Constraints: Generalized Armstrong: with k = digit count, sum of (each digit^k) equals n.

Sample Input: 153

Expected Output: Yes

Judge test case count: 5

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1

Expected output:

Yes

Case 2 (public)**Input:**

153

Expected output:

Yes

Case 3 (public)**Input:**

9474

Expected output:

Yes

Case 4 (public)**Input:**

200

Expected output:

No

Case 5 (public)**Input:**

123

Expected output:

No

6. Phase 1 — Foundation — Introduction to C — Q6: Read Name And Age Greeting

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Read Name And Age Greeting. Read input from stdin and print exact output.

Input Format: Line 1: name string. Line 2: integer age.

Output Format: Print one greeting sentence exactly.

Constraints: Person gives one word nickname, then a normal human age.

Sample Input:

Alex

21

Expected Output: Hello Alex, you are 21 years old.

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Amy

17

Expected output:

Hello Amy, you are 17 years old.

Case 2 (public)

Input:

Leo

0

Expected output:

Hello Leo, you are 0 years old.

Case 3 (public)

Input:

Zoe

100

Expected output:

Hello Zoe, you are 100 years old.

7. Phase 1 — Foundation — Introduction to C — Q7: Array Reverse Print

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Array Reverse Print. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print elements in reverse order, space-separated.

Constraints: `n` then list length n ints.

Sample Input:

4

1 2 3 4

Expected Output: 4 3 2 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

42

Expected output:

42

Case 2 (public)

Input:

2

0 5

Expected output:

5 0

Case 3 (public)

Input:

5

1 1 2 3 5

Expected output:

5 3 2 1 1

8. Phase 1 — Foundation — Introduction to C — Q8: Student Record Variables

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Student Record Variables. Read input from stdin and print exact output.

Input Format: Four lines: name, age, grade letter, tuition fees.

Output Format: Print four fields separated by spaces: name age grade fees.

Constraints: Four separate answers: word name, numeric age, one letter grade, tuition digits.

Sample Input:

Mia

19

B

72000

Expected Output: Mia 19 B 72000

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Raj

22

C

61000

Expected output:

Raj 22 C 61000

Case 2 (public)

Input:

Zoe

18

F

8000

Expected output:

Zoe 18 F 8000

Case 3 (public)

Input:

Leo

30

D

41234

Expected output:

Leo 30 D 41234

9. Phase 1 — Foundation — Introduction to C — Q9: Break On Negative Sum

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Break On Negative Sum. Read input from stdin and print exact output.

Input Format: First line count n, then n integers one per line (last may be negative).

Output Format: Print sum of values before first negative integer.

Constraints: Count prefixed then sequence entries until negative sentinel occurs.

Sample Input:

3
5
8
-2

Expected Output: 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4
1
2
3
-9

Expected output:

6

Case 2 (public)

Input:

1
-5

Expected output:

0

Case 3 (public)

Input:

5
2
2
2
2
-1

Expected output:

8

10. Phase 1 — Foundation — Introduction to C — Q10: Swap Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Introduction to C'.

Problem style: Swap Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped values: b a.

Constraints: Two integers arrive in order.

Sample Input: 7 3

Expected Output: 3 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2

Expected output:

2 1

Case 2 (public)

Input:

9 9

Expected output:

9 9

Case 3 (public)

Input:

-5 8

Expected output:

8 -5

11. Phase 1 — Foundation — Structure of a C Program — Q11: Bubble Sort Array

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Bubble Sort Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Print sorted ascending, space-separated (bubble sort / any stable $O(n^2)$ ok).

Constraints: `n` moderate sortable ints.

Sample Input:

4

4 1 3 2

Expected Output: 1 2 3 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

2 1

Expected output:

1 2

Case 2 (public)

Input:

5

5 4 3 2 1

Expected output:

1 2 3 4 5

Case 3 (public)

Input:

1

7

Expected output:

7

12. Phase 1 — Foundation — Structure of a C Program — Q12: Calculator Switch Menu

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Calculator Switch Menu. Read input from stdin and print exact output.

Input Format: Line 1: op. Line 2: a. Line 3: b. Integer division for '/'.

Output Format: Print integer result.

Constraints: Operator letter then two operands lines.

Sample Input:

+

4

11

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

+
10
5

Expected output:

15

Case 2 (public)

Input:

/
49
7

Expected output:

7

Case 3 (public)

Input:

*
6
7

Expected output:

42

13. Phase 1 — Foundation — Structure of a C Program — Q13: Check Palindrome

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Check Palindrome. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print Yes if palindrome, else No.

Constraints: One string with no gaps.

Sample Input: madam

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

Yes

Case 2 (public)

Input:

abba

Expected output:

Yes

Case 3 (public)

Input:

abc

Expected output:

No

14. Phase 1 — Foundation — Structure of a C Program — Q14: Concatenate Without Strcat

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Concatenate Without Strcat. Read input from stdin and print exact output.

Input Format: Line 1 string A; line 2 string B.

Output Format: Print A+B exactly.

Constraints: Two textual tokens sequentially lines.

Sample Input:

code

lab

Expected Output: codelab

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

x

yz

Expected output:

xyz

Case 2 (public)

Input:

tree

House

Expected output:

treeHouse

Case 3 (public)

Input:

a

b

Expected output:

ab

15. Phase 1 — Foundation — Structure of a C Program — Q15: Read Float Decimal Places

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Read Float Decimal Places. Read input from stdin and print exact output.

Input Format: One float on one line.

Output Format: Print three values with 2, 4, then 6 decimal places, space-separated.

Constraints: One fractional number shown with decimal digits.

Sample Input: 3.14159

Expected Output: 3.14 3.1416 3.141590

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9.87

Expected output:

9.87 9.8700 9.870000

Case 2 (public)

Input:

0.1

Expected output:

0.10 0.1000 0.100000

Case 3 (public)

Input:

2.5

Expected output:

2.50 2.5000 2.500000

16. Phase 1 — Foundation — Structure of a C Program — Q16: Read Character Ascii

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Read Character Ascii. Read input from stdin and print exact output.

Input Format: A single character on one line.

Output Format: Print its decimal ASCII code.

Constraints: One visible character keyed by user.

Sample Input: A

Expected Output: 65

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

48

Case 2 (public)

Input:

9

Expected output:

57

Case 3 (public)

Input:

@

Expected output:

64

17. Phase 1 — Foundation — Structure of a C Program — Q17: Student Record Variables

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Student Record Variables. Read input from stdin and print exact output.

Input Format: Four lines: name, age, grade letter, tuition fees.

Output Format: Print four fields separated by spaces: name age grade fees.

Constraints: Four separate answers: word name, numeric age, one letter grade, tuition digits.

Sample Input:

Mia

19

B

72000

Expected Output: Mia 19 B 72000

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Raj

22

C

61000

Expected output:

Raj 22 C 61000

Case 2 (public)

Input:

Zoe
18
F
8000

Expected output:

Zoe 18 F 8000

Case 3 (public)**Input:**

Leo
30
D
41234

Expected output:

Leo 30 D 41234

18. Phase 1 — Foundation — Structure of a C Program — Q18: Print Data Type Sizes Lab

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Print Data Type Sizes Lab. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print four integers separated by spaces: sizeof(int) sizeof(float) sizeof(double) sizeof(unsigned char).

Constraints: No input values (lab assumes fixed widths for grading).

Sample Input:

Expected Output: 4 4 8 1

Judge test case count: 1

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

(empty)

Expected output:

4 4 8 1

19. Phase 1 — Foundation — Structure of a C Program — Q19: Print Simple Box Border

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Print Simple Box Border. Read input from stdin and print exact output.

Input Format: One integer n (side length of hollow square drawn with '*'). Middle uses spaces.

Output Format: Print exactly n rows: rows 1 and n are n stars; rows 2..n-1 are star, (n-2 spaces), star. Each line ends right after the last character—no trailing spaces.

Constraints: Border size decides rows/columns counts for hollow square artwork.

Sample Input: 3

Expected Output (sample):

* *

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

3

Expected output:

* * *

* * *

* * *

Case 2 (public)

Input:

4

Expected output:

* * * *

* * *

* * *

* * * *

Case 3 (public)

Input:

5

Expected output:

* * * * *

* * *

* * *

* * *

* * * * *

20. Phase 1 — Foundation — Structure of a C Program — Q20: Swap Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Structure of a C Program'.

Problem style: Swap Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped values: b a.

Constraints: Two integers arrive in order.

Sample Input: 7 3

Expected Output: 3 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2

Expected output:

2 1

Case 2 (public)

Input:

9 9

Expected output:

9 9

Case 3 (public)

Input:

-5 8

Expected output:

8 -5

21. Phase 1 — Foundation — Keywords and Identifiers — Q21: Copy File

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Copy File. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Print the same text (simulating copy output).

Constraints: One line echoed as duplicated output.

Sample Input: copy this

Expected Output: copy this

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

mno

Expected output:

mno

Case 2 (public)

Input:

data

Expected output:

data

Case 3 (public)

Input:

done

Expected output:

done

22. Phase 1 — Foundation — Keywords and Identifiers — Q22: Count Above Average

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Count Above Average. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Count elements strictly greater than floor(average); average = floor(sum/n).

Constraints: Array ints general distribution.

Sample Input:

5

2 2 2 2 8

Expected Output: 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3

10 10 10

Expected output:

0

Case 2 (public)

Input:

4

9 10 11 100

Expected output:

1

Case 3 (public)

Input:

2

5 15

Expected output:

1

23. Phase 1 — Foundation — Keywords and Identifiers — Q23: Count Vowels

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Count Vowels. Read input from stdin and print exact output.

Input Format: One lowercase/uppercase string.

Output Format: Print vowel count (a, e, i, o, u).

Constraints: One line mix of uppercase/lowercase letters.

Sample Input: education

Expected Output: 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

AEIOU

Expected output:

5

Case 2 (public)

Input:

programming

Expected output:

3

Case 3 (public)

Input:

bcd

Expected output:

0

24. Phase 1 — Foundation — Keywords and Identifiers — Q24: Count Vowels Consonants Spaces

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Count Vowels Consonants Spaces. Read input from stdin and print exact output.

Input Format: One line: sentence.

Output Format: Print three integers: vowels (aeiou AEIOU), consonants (other letters), spaces.

Constraints: One sentence letters spaces only simplifying classification.

Sample Input: Hi there

Expected Output: 3 4 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

ABC

Expected output:

1 2 0

Case 2 (public)

Input:

a b c

Expected output:

1 2 2

Case 3 (public)

Input:

rhythm

Expected output:

0 6 0

25. Phase 1 — Foundation — Keywords and Identifiers — Q25: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

26. Phase 1 — Foundation — Keywords and Identifiers — Q26: Swap Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Swap Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped values: b a.

Constraints: Two integers arrive in order.

Sample Input: 7 3

Expected Output: 3 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2

Expected output:

2 1

Case 2 (public)

Input:

9 9

Expected output:

9 9

Case 3 (public)

Input:

-5 8

Expected output:

27. Phase 1 — Foundation — Keywords and Identifiers — Q27: Check Even/Odd

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Check Even/Odd. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Even if n is even, else Odd.

Constraints: One integer (can be negative or zero).

Sample Input: 8

Expected Output: Even

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

0

Expected output:

Even

Case 2 (public)

Input:

-3

Expected output:

Odd

Case 3 (public)

Input:

14

Expected output:

Even

28. Phase 1 — Foundation — Keywords and Identifiers — Q28: Find Largest Of 3 Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Find Largest Of 3 Numbers. Read input from stdin and print exact output.

Input Format: Three integers a, b, c.

Output Format: Print the largest value.

Constraints: Three integers separated on one line (or consecutive reads).

Sample Input: 9 2 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

1 2 3

Expected output:

3

Case 2 (public)

Input:

5 5 1

Expected output:

5

Case 3 (public)

Input:

7 4 7

Expected output:

7

29. Phase 1 — Foundation — Keywords and Identifiers — Q29: Grade Calculator

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Keywords and Identifiers'.

Problem style: Grade Calculator. Read input from stdin and print exact output.

Input Format: One integer marks.

Output Format: Print grade: A (≥ 90), B (≥ 80), C (≥ 70), D (≥ 60), F (< 60).

Constraints: One mark from 0 to 100.

Sample Input: 82

Expected Output: B

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

74

Expected output:

C

Case 2 (public)

Input:

60

Expected output:

D

Case 3 (public)

Input:

48

Expected output:

F

30. Phase 1 — Foundation — Variables — Q30: Employee Details

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Employee Details. Read input from stdin and print exact output.

Input Format: n followed by n lines: id name.

Output Format: Print employee details as id name per line.

Constraints: Count then lines with id integer and name.

Sample Input:

2

101 Alice

102 Bob

Expected Output (sample):

101 Alice

102 Bob

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

301 Mia

Expected output:

301 Mia

Case 2 (public)

Input:

2
11 Raj
12 Dee

Expected output:

11 Raj
12 Dee

Case 3 (public)**Input:**

3
1 X
2 Y
3 Z

Expected output:

1 X
2 Y
3 Z

31. Phase 1 — Foundation — Variables — Q31: Equilateral Star Pattern

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Equilateral Star Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows in the centered star triangle).

Output Format: For each row r from 0 to n-1 (top to bottom): print exactly (n-1-r) spaces, then (r+1) stars with a single space between consecutive stars, and no space after the last star on that line. This centers each row so column alignment matches the sample for every test case.

Constraints: Same spacing rule for every row: row r uses (n-1-r) spaces then r+1 stars with single spaces between stars; no trailing spaces.

Sample Input: 4

Expected Output (sample):

```
*  
* *  
* * *  
* * * *
```

Judge test case count: 4

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1

Expected output:

*

Case 2 (public)**Input:**

2

Expected output:

```
*  
* *
```

Case 3 (public)**Input:**

3

Expected output:

```
*  
* *  
* * *
```

Case 4 (public)

Input:

4

Expected output:

```
*
* *
* * *
* * * *
```

32. Phase 1 — Foundation — Variables — Q32: Factorial Of Number

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Factorial Of Number. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print n!.

Constraints: Small non-negative integer n (factorial grows fast).

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

6

Expected output:

720

Case 3 (public)

Input:

3

Expected output:

6

33. Phase 1 — Foundation — Variables — Q33: Factorial Using Function

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Factorial Using Function. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print factorial of n using a function.

Constraints: Same as factorial puzzle but enforce your own helper.

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

1

Case 2 (public)

Input:

1

Expected output:

1

Case 3 (public)

Input:

6

Expected output:

720

34. Phase 1 — Foundation — Variables — Q34: Swap Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Swap Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped values: b a.

Constraints: Two integers arrive in order.

Sample Input: 7 3

Expected Output: 3 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2

Expected output:

2 1

Case 2 (public)

Input:

9 9

Expected output:

9 9

Case 3 (public)

Input:

-5 8

Expected output:

8 -5

35. Phase 1 — Foundation — Variables — Q35: Read Name And Age Greeting

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Read Name And Age Greeting. Read input from stdin and print exact output.

Input Format: Line 1: name string. Line 2: integer age.

Output Format: Print one greeting sentence exactly.

Constraints: Person gives one word nickname, then a normal human age.

Sample Input:

Alex

21

Expected Output: Hello Alex, you are 21 years old.

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Amy

17

Expected output:

Hello Amy, you are 17 years old.

Case 2 (public)

Input:

Leo

0

Expected output:

Hello Leo, you are 0 years old.

Case 3 (public)

Input:

Zoe

100

Expected output:

Hello Zoe, you are 100 years old.

36. Phase 1 — Foundation — Variables — Q36: Print Name And City

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Print Name And City. Read input from stdin and print exact output.

Input Format: Line 1: name. Line 2: city.

Output Format: Print name on line 1, city on line 2.

Constraints: You get two answers the user typed: first the name word, second the city word — each alone on its line.

Sample Input:

Riya

Chennai

Expected Output (sample):

Riya

Chennai

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Ada

Boston

Expected output:

Ada

Boston

Case 2 (public)

Input:

Riya

Chennai

Expected output:

Riya

Chennai

Case 3 (public)

Input:

Zoe

Paris

Expected output:

Zoe

Paris

37. Phase 1 — Foundation — Variables — Q37: Read Float Decimal Places

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Read Float Decimal Places. Read input from stdin and print exact output.

Input Format: One float on one line.

Output Format: Print three values with 2, 4, then 6 decimal places, space-separated.

Constraints: One fractional number shown with decimal digits.

Sample Input: 3.14159

Expected Output: 3.14 3.1416 3.141590

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9.87

Expected output:

9.87 9.8700 9.870000

Case 2 (public)

Input:

0.1

Expected output:

0.10 0.1000 0.100000

Case 3 (public)

Input:

2.5

Expected output:

2.50 2.5000 2.500000

38. Phase 1 — Foundation — Variables — Q38: Read Character Ascii

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Variables'.

Problem style: Read Character Ascii. Read input from stdin and print exact output.

Input Format: A single character on one line.

Output Format: Print its decimal ASCII code.

Constraints: One visible character keyed by user.

Sample Input: A

Expected Output: 65

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

48

Case 2 (public)

Input:

9

Expected output:

57

Case 3 (public)

Input:

@

Expected output:

64

39. Phase 1 — Foundation — Data Types — Q39: Fibonacci Series

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Fibonacci Series. Read input from stdin and print exact output.

Input Format: One integer n (number of terms).

Output Format: Print first n Fibonacci numbers separated by spaces.

Constraints: Count of terms to emit.

Sample Input: 7

Expected Output: 0 1 1 2 3 5 8

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

0 1

Case 2 (public)

Input:

3

Expected output:

0 1 1

Case 3 (public)

Input:

6

Expected output:

0 1 1 2 3 5

40. Phase 1 — Foundation — Data Types — Q40: Find Sum Of Array

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Find Sum Of Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of elements.

Constraints: First n, then exactly n integers on the next input line.

Sample Input:

5

1 2 3 4 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

10

Expected output:

10

Case 2 (public)

Input:

4

0 0 0 0

Expected output:

0

Case 3 (public)

Input:

3
7 8 9

Expected output:

24

41. Phase 1 — Foundation — Data Types — Q41: Floyd Triangle Pattern

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Floyd Triangle Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows).

Output Format: Floyd triangle: row i has i consecutive integers, numbers separated by a single space; rows separated by newline. Each line ends immediately after the last digit—no trailing spaces.

Constraints: Height rows grows consecutive integers pattern.

Sample Input: 3

Expected Output (sample):

1
2 3
4 5 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

4

Expected output:

1
2 3
4 5 6
7 8 9 10

Case 3 (public)

Input:

3

Expected output:

1
2 3
4 5 6

42. Phase 1 — Foundation — Data Types — Q42: Gcd Euclidean

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Gcd Euclidean. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print GCD(a,b).

Constraints: Two positive integers on one line (space-separated), same as typical scanf pair input.

Sample Input: 48 18

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

7 13

Expected output:

1

Case 2 (public)

Input:

54 24

Expected output:

6

Case 3 (public)

Input:

17 17

Expected output:

17

43. Phase 1 — Foundation — Data Types — Q43: Read Character Ascii

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Read Character Ascii. Read input from stdin and print exact output.

Input Format: A single character on one line.

Output Format: Print its decimal ASCII code.

Constraints: One visible character keyed by user.

Sample Input: A

Expected Output: 65

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

48

Case 2 (public)

Input:

9

Expected output:

57

Case 3 (public)

Input:

@

Expected output:

64

44. Phase 1 — Foundation — Data Types — Q44: Read Float Decimal Places

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Read Float Decimal Places. Read input from stdin and print exact output.

Input Format: One float on one line.

Output Format: Print three values with 2, 4, then 6 decimal places, space-separated.

Constraints: One fractional number shown with decimal digits.

Sample Input: 3.14159

Expected Output: 3.14 3.1416 3.141590

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9.87

Expected output:

9.87 9.8700 9.870000

Case 2 (public)

Input:

0.1

Expected output:

0.10 0.1000 0.100000

Case 3 (public)

Input:

2.5

Expected output:

2.50 2.5000 2.500000

45. Phase 1 — Foundation — Data Types — Q45: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

46. Phase 1 — Foundation — Data Types — Q46: Check Even/Odd

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Check Even/Odd. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Even if n is even, else Odd.

Constraints: One integer (can be negative or zero).

Sample Input: 8

Expected Output: Even

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Even

Case 2 (public)

Input:

-3

Expected output:

Odd

Case 3 (public)

Input:

14

Expected output:

Even

47. Phase 1 — Foundation — Data Types — Q47: Print Hello World

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Print Hello World. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print exactly: Hello World

Constraints: This program runs without waiting for typed input.

Sample Input:

Expected Output: Hello World

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

Hello world

48. Phase 1 — Foundation — Data Types — Q48: Print Data Type Sizes Lab

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Data Types'.

Problem style: Print Data Type Sizes Lab. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print four integers separated by spaces: sizeof(int) sizeof(float) sizeof(double) sizeof(unsigned char).

Constraints: No input values (lab assumes fixed widths for grading).

Sample Input:

Expected Output: 4 4 8 1

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

4 4 8 1

49. Phase 1 — Foundation — Constants — Q49: Is Even Function Loop

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Is Even Function Loop. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print all even numbers from 2 to n inclusive, space-separated.

Constraints: Upper bound counting evens inclusively downward.

Sample Input: 10

Expected Output: 2 4 6 8 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

(empty)

Case 2 (public)

Input:

4

Expected output:

2 4

Case 3 (public)

Input:

12

Expected output:

2 4 6 8 10 12

50. Phase 1 — Foundation — Constants — Q50: Largest Element

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Largest Element. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print largest array element.

Constraints: n then n integers.

Sample Input:

5

1 2 9 4 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

Expected output:

7

Case 2 (public)

Input:

4

2 2 2 2

Expected output:

2

Case 3 (public)**Input:**

5

8 1 3 6 4

Expected output:

8

51. Phase 1 — Foundation — Constants — Q51: Lcm Using Gcd

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Lcm Using Gcd. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print LCM(a,b) using $LCM = a*b/GCD(a,b)$ with integer arithmetic.

Constraints: Two integers on one line (space-separated).

Sample Input: 4 6

Expected Output: 12

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

3 9

Expected output:

9

Case 2 (public)**Input:**

8 12

Expected output:

24

Case 3 (public)**Input:**

11 13

Expected output:

143

52. Phase 1 — Foundation — Constants — Q52: Linear Search Array

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Linear Search Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers. Line 3: target.

Output Format: Print 0-based index of first occurrence, or -1 if not found.

Constraints: Array listing plus target finder.

Sample Input:

5

1 4 7 9 2

7

Expected Output: 2

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

4

8 8 8 8

8

Expected output:

0

Case 2 (public)

Input:

2

1 2

3

Expected output:

-1

Case 3 (public)

Input:

1

42

42

Expected output:

0

53. Phase 1 — Foundation — Constants — Q53: Print Hello World

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Print Hello World. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print exactly: Hello World

Constraints: This program runs without waiting for typed input.

Sample Input:

Expected Output: Hello World

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

Hello world

54. Phase 1 — Foundation — Constants — Q54: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

55. Phase 1 — Foundation — Constants — Q55: Read Character Ascii

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Read Character Ascii. Read input from stdin and print exact output.

Input Format: A single character on one line.

Output Format: Print its decimal ASCII code.

Constraints: One visible character keyed by user.

Sample Input: A

Expected Output: 65

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

48

Case 2 (public)

Input:

9

Expected output:

57

Case 3 (public)

Input:

@

Expected output:

64

56. Phase 1 — Foundation — Constants — Q56: Read Float Decimal Places

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Read Float Decimal Places. Read input from stdin and print exact output.

Input Format: One float on one line.

Output Format: Print three values with 2, 4, then 6 decimal places, space-separated.

Constraints: One fractional number shown with decimal digits.

Sample Input: 3.14159

Expected Output: 3.14 3.1416 3.141590

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9.87

Expected output:

9.87 9.8700 9.870000

Case 2 (public)

Input:

0.1

Expected output:

0.10 0.1000 0.100000

Case 3 (public)

Input:

2.5

Expected output:

2.50 2.5000 2.500000

57. Phase 1 — Foundation — Constants — Q57: Print Name And City

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Constants'.

Problem style: Print Name And City. Read input from stdin and print exact output.

Input Format: Line 1: name. Line 2: city.

Output Format: Print name on line 1, city on line 2.

Constraints: You get two answers the user typed: first the name word, second the city word — each alone on its line.

Sample Input:

Riya

Chennai

Expected Output (sample):

Riya

Chennai

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Ada

Boston

Expected output:

Ada

Boston

Case 2 (public)

Input:

Riya

Chennai

Expected output:

Riya

Chennai

Case 3 (public)

Input:

Zoe

Paris

Expected output:

Zoe

Paris

58. Phase 1 — Foundation — Header Files — Q58: Matrix Addition

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Matrix Addition. Read input from stdin and print exact output.

Input Format: r c, then matrix A (r lines), then matrix B (r lines).

Output Format: Print A+B matrix row-wise.

Constraints: Dimensions plus two matrices stacked in stdin order.

Sample Input:

```
2 2
1 2
3 4
5 6
7 8
```

Expected Output (sample):

```
6 8
10 12
```

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1 1
2
7
```

Expected output:

```
9
```

Case 2 (public)

Input:

```
2 2
1 1
1 1
2 2
2 2
```

Expected output:

```
3 3
3 3
```

Case 3 (public)

Input:

```
1 3
1 2 3
3 2 1
```

Expected output:

```
4 4 4
```

59. Phase 1 — Foundation — Header Files — Q59: Matrix Print 3X3

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Matrix Print 3X3. Read input from stdin and print exact output.

Input Format: Three lines, each with three integers.

Output Format: Print matrix row-wise with spaces between numbers per row.

Constraints: Exactly nine ints row major three lines typed.

Sample Input:

```
1 2 3
4 5 6
7 8 9
```

Expected Output (sample):

```
1 2 3
4 5 6
7 8 9
```

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)**Input:**

```
9 8 7
6 5 4
3 2 1
```

Expected output:

```
9 8 7
6 5 4
3 2 1
```

Case 2 (public)**Input:**

```
1 0 0
0 1 0
0 0 1
```

Expected output:

```
1 0 0
0 1 0
0 0 1
```

60. Phase 1 — Foundation — Header Files — Q60: Matrix Row Column Sums

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Matrix Row Column Sums. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Line 1: row sums space-separated. Line 2: column sums space-separated.

Constraints: 3×3 numeric grid unchanged.

Sample Input:

```
1 2 3
4 5 6
7 8 9
```

Expected Output (sample):

```
6 15 24
12 15 18
```

Judge test case count: 2

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

```
1 2 3
4 5 6
7 8 9
```

Expected output:

```
6 15 24
12 15 18
```

Case 2 (public)**Input:**

```
0 0 1
0 1 0
1 0 0
```

Expected output:

```
1 1 1
1 1 1
```

61. Phase 1 — Foundation — Header Files — Q61: Matrix Transpose

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Matrix Transpose. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Print transpose (3 lines).

Constraints: 3×3 square matrix.

Sample Input:

1 2 3

4 5 6

7 8 9

Expected Output (sample):

1 4 7

2 5 8

3 6 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3

0 0 0

7 8 9

Expected output:

1 0 7

2 0 8

3 0 9

Case 2 (public)

Input:

1 2 3

4 5 6

7 8 9

Expected output:

1 4 7

2 5 8

3 6 9

Case 3 (public)

Input:

2 0 1

0 3 0

1 0 2

Expected output:

2 0 1

0 3 0

1 0 2

62. Phase 1 — Foundation — Header Files — Q62: Print Data Type Sizes Lab

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Print Data Type Sizes Lab. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print four integers separated by spaces: sizeof(int) sizeof(float) sizeof(double) sizeof(unsigned char).

Constraints: No input values (lab assumes fixed widths for grading).

Sample Input:

Expected Output: 4 4 8 1

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

4 4 8 1

63. Phase 1 — Foundation — Header Files — Q63: Print Hello World

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Print Hello World. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print exactly: Hello World

Constraints: This program runs without waiting for typed input.

Sample Input:

Expected Output: Hello World

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

Hello world

64. Phase 1 — Foundation — Header Files — Q64: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

65. Phase 1 — Foundation — Header Files — Q65: Print Simple Box Border

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Print Simple Box Border. Read input from stdin and print exact output.

Input Format: One integer n (side length of hollow square drawn with '*'). Middle uses spaces.

Output Format: Print exactly n rows: rows 1 and n are n stars; rows 2..n-1 are star, (n-2 spaces), star. Each line ends right after the last character—no trailing spaces.

Constraints: Border size decides rows/columns counts for hollow square artwork.

Sample Input: 3

Expected Output (sample):

```
***
```

```
*  *
```

```
***
```

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3

Expected output:

```
***
```

```
*  *
```

```
***
```

Case 2 (public)

Input:

4

Expected output:

```
****
```

```
*  *
```

```
*  *
```

```
****
```

Case 3 (public)

Input:

5

Expected output:

```
*****
```

```
*      *
```

```
*      *
```

```
*      *
```

```
*****
```

66. Phase 1 — Foundation — Header Files — Q66: Pattern Printing

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Header Files'.

Problem style: Pattern Printing. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print a right-angle star pattern with n lines (only the characters shown—no trailing spaces after the last character on each line).

Constraints: One height n determines how many lines of stars.

Sample Input: 3

Expected Output (sample):

```
*
```

```
**
```

```
***
```

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

*

**

Case 2 (public)

Input:

4

Expected output:

*

**

Case 3 (public)

Input:

5

Expected output:

*

**

67. Phase 1 — Foundation — Input and Output (printf, scanf) — Q67: Max Min Average Array

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Max Min Average Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print max min average as integers: $\text{average} = \text{floor}(\text{sum}/n)$.

Constraints: `n` then n ints manageable array sizes.

Sample Input:

4

2 8 4 6

Expected Output: 8 2 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

Expected output:

7 7 7

Case 2 (public)

Input:

2

1 2

Expected output:

2 1 1

Case 3 (public)

Input:

5

1 1 1 1 1

Expected output:

1 1 1

68. Phase 1 — Foundation — Input and Output (printf, scanf) — Q68: Palindrome Number

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Palindrome Number. Read input from stdin and print exact output.

Input Format: One integer n (no leading zeros except single 0).

Output Format: Print Yes if decimal digits read same forward/backward, else No.

Constraints: Integer decimal digit symmetry checking.

Sample Input: 121

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

0

Expected output:

Yes

Case 2 (public)

Input:

1221

Expected output:

Yes

Case 3 (public)

Input:

10

Expected output:

No

69. Phase 1 — Foundation — Input and Output (printf, scanf) — Q69: Pascal Triangle Rows

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Pascal Triangle Rows. Read input from stdin and print exact output.

Input Format: One integer n = number of rows.

Output Format: Print Pascal triangle: row i has i integers, space-separated.

Constraints: `n` rows modest combinator growth.

Sample Input: 3

Expected Output (sample):

1

1 1

1 2 1

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

1

Expected output:

1

Case 2 (public)

Input:

2

Expected output:

1

1 1

Case 3 (public)

Input:

5

Expected output:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

70. Phase 1 — Foundation — Input and Output (printf, scanf) — Q70: Pointer To Pointer

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Pointer To Pointer. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print value using pointer-to-pointer dereference.

Constraints: One integer echoed conceptually.

Sample Input: 7

Expected Output: 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

0

Case 2 (public)

Input:

15

Expected output:

15

Case 3 (public)

Input:

-8

Expected output:

-8

71. Phase 1 — Foundation — Input and Output (printf, scanf) — Q71: Print Name And City

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Print Name And City. Read input from stdin and print exact output.

Input Format: Line 1: name. Line 2: city.

Output Format: Print name on line 1, city on line 2.

Constraints: You get two answers the user typed: first the name word, second the city word — each alone on its line.

Sample Input:

Riya

Chennai

Expected Output (sample):

Riya

Chennai

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Ada
Boston

Expected output:

Ada
Boston

Case 2 (public)

Input:

Riya
Chennai

Expected output:

Riya
Chennai

Case 3 (public)

Input:

Zoe
Paris

Expected output:

Zoe
Paris

72. Phase 1 — Foundation — Input and Output (printf, scanf) — Q72: Read Float Decimal Places

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Read Float Decimal Places. Read input from stdin and print exact output.

Input Format: One float on one line.

Output Format: Print three values with 2, 4, then 6 decimal places, space-separated.

Constraints: One fractional number shown with decimal digits.

Sample Input: 3.14159

Expected Output: 3.14 3.1416 3.141590

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9.87

Expected output:

9.87 9.8700 9.870000

Case 2 (public)

Input:

0.1

Expected output:

0.10 0.1000 0.100000

Case 3 (public)

Input:

2.5

Expected output:

2.50 2.5000 2.500000

73. Phase 1 — Foundation — Input and Output (printf, scanf) — Q73: Read Name And Age Greeting

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Read Name And Age Greeting. Read input from stdin and print exact output.

Input Format: Line 1: name string. Line 2: integer age.

Output Format: Print one greeting sentence exactly.

Constraints: Person gives one word nickname, then a normal human age.

Sample Input:

Alex

21

Expected Output: Hello Alex, you are 21 years old.

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Amy

17

Expected output:

Hello Amy, you are 17 years old.

Case 2 (public)

Input:

Leo

0

Expected output:

Hello Leo, you are 0 years old.

Case 3 (public)

Input:

Zoe

100

Expected output:

Hello Zoe, you are 100 years old.

74. Phase 1 — Foundation — Input and Output (printf, scanf) — Q74: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

- 7

75. Phase 1 — Foundation — Input and Output (printf, scanf) — Q75: Read Character Ascii

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Input and Output (printf, scanf)'.

Problem style: Read Character Ascii. Read input from stdin and print exact output.

Input Format: A single character on one line.

Output Format: Print its decimal ASCII code.

Constraints: One visible character keyed by user.

Sample Input: A

Expected Output: 65

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

0

Expected output:

48

Case 2 (public)

Input:

9

Expected output:

57

Case 3 (public)

Input:

@

Expected output:

64

76. Phase 1 — Foundation — Compilation process — Q76: Prime Check

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Prime Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print exactly Prime or Not (capital P / capital N as shown).

Constraints: Any integer: $n < 2$ and negatives are Not; primes are ≥ 2 with no nontrivial divisor.

Sample Input: 7

Expected Output: Prime

Judge test case count: 6

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

2

Expected output:

Prime

Case 2 (public)

Input:

9

Expected output:

Not

Case 3 (public)

Input:

17

Expected output:

Prime

Case 4 (public)

Input:

1

Expected output:

Not

Case 5 (public)

Input:

0

Expected output:

Not

Case 6 (public)

Input:

-11

Expected output:

Not

77. Phase 1 — Foundation — Compilation process — Q77: Print Primes In Range

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Print Primes In Range. Read input from stdin and print exact output.

Input Format: One line with two integers lo and hi separated by whitespace.

Output Format: Print all primes in [lo, hi] inclusive, space-separated, ascending.

Constraints: Two bounds lo and hi on one line (space-separated), inclusive range.

Sample Input: 5 15

Expected Output: 5 7 11 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

2

Case 2 (public)

Input:

14 18

Expected output:

17

Case 3 (public)

Input:

20 30

Expected output:

23 29

78. Phase 1 — Foundation — Compilation process — Q78: Product Inventory Record

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Product Inventory Record. Read input from stdin and print exact output.

Input Format: n followed by n lines: product_code quantity.

Output Format: Print each product and quantity on new line.

Constraints: Count then product code strings with quantity.

Sample Input:

2
P1 10
P2 5

Expected Output (sample):

P1 10
P2 5

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1
B2 10

Expected output:

B2 10

Case 2 (public)**Input:**

2
C1 1
C2 2

Expected output:

C1 1
C2 2

Case 3 (public)**Input:**

3
D1 9
D2 8
D3 7

Expected output:

D1 9
D2 8
D3 7

79. Phase 1 — Foundation — Compilation process — Q79: Read File Content

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Read File Content. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Print the given text (simulating file content read).

Constraints: One line echoed as-if read from disk.

Sample Input: sample content

Expected Output: sample content

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

x

Expected output:

x

Case 2 (public)**Input:**

read me

Expected output:

read me

Case 3 (public)

Input:

42

Expected output:

42

80. Phase 1 — Foundation — Compilation process — Q80: Reverse A String

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Reverse A String. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print reversed string.

Constraints: One word/token without spaces (matching how the checker reads input).

Sample Input: hello

Expected Output: olleh

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

a

Case 2 (public)

Input:

racecar

Expected output:

racecar

Case 3 (public)

Input:

coder

Expected output:

redoc

81. Phase 1 — Foundation — Compilation process — Q81: Print Hello World

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Print Hello World. Read input from stdin and print exact output.

Input Format: No input.

Output Format: Print exactly: Hello World

Constraints: This program runs without waiting for typed input.

Sample Input:

Expected Output: Hello World

Judge test case count: 1

All test cases (for evaluation / marking):

Case 1 (public)

Input:

(empty)

Expected output:

Hello world

82. Phase 1 — Foundation — Compilation process — Q82: Add Two Numbers

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

83. Phase 1 — Foundation — Compilation process — Q83: Student Record Variables

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Student Record Variables. Read input from stdin and print exact output.

Input Format: Four lines: name, age, grade letter, tuition fees.

Output Format: Print four fields separated by spaces: name age grade fees.

Constraints: Four separate answers: word name, numeric age, one letter grade, tuition digits.

Sample Input:

Mia

19

B

72000

Expected Output: Mia 19 B 72000

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Raj

22

C

61000

Expected output:

Raj 22 C 61000

Case 2 (public)

Input:

Zoe

18

F

8000

Expected output:

Zoe 18 F 8000

Case 3 (public)

Input:

Leo

30

D

41234

Expected output:

Leo 30 D 41234

84. Phase 1 — Foundation — Compilation process — Q84: Print Simple Box Border

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Print Simple Box Border. Read input from stdin and print exact output.

Input Format: One integer n (side length of hollow square drawn with '*'). Middle uses spaces.

Output Format: Print exactly n rows: rows 1 and n are n stars; rows 2..n-1 are star, (n-2 spaces), star. Each line ends right after the last character—no trailing spaces.

Constraints: Border size decides rows/columns counts for hollow square artwork.

Sample Input: 3

Expected Output (sample):

* *

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3

Expected output:

* *

Case 2 (public)

Input:

4

Expected output:

* *

* *

Case 3 (public)

Input:

5

Expected output:

* *

* *

* *

85. Phase 1 — Foundation — Compilation process — Q85: Pattern Printing

Module: Phase 1 — Foundation

Difficulty: easy

Description: Solve a phase 1 — foundation coding task focused on 'Compilation process'.

Problem style: Pattern Printing. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print a right-angle star pattern with n lines (only the characters shown—no trailing spaces after the last character on each line).

Constraints: One height n determines how many lines of stars.

Sample Input: 3

Expected Output (sample):

```
*
**
***
```

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

```
*
**
```

Case 2 (public)

Input:

4

Expected output:

```
*
**
***
****
```

Case 3 (public)

Input:

5

Expected output:

```
*
**
***
****
*****
```

86. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q1: Check Even/Odd

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Check Even/Odd. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Even if n is even, else Odd.

Constraints: One integer (can be negative or zero).

Sample Input: 8

Expected Output: Even

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Even

Case 2 (public)

Input:

-3

Expected output:

Odd

Case 3 (public)

Input:

14

Expected output:

Even

87. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q2: Find Largest Of 3 Numbers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Find Largest Of 3 Numbers. Read input from stdin and print exact output.

Input Format: Three integers a, b, c.

Output Format: Print the largest value.

Constraints: Three integers separated on one line (or consecutive reads).

Sample Input: 9 2 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3

Expected output:

3

Case 2 (public)

Input:

5 5 1

Expected output:

5

Case 3 (public)

Input:

7 4 7

Expected output:

7

88. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q3: Second Largest Array

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Second Largest Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Compute M = maximum element. If some array value is smaller than M, print the largest among values strictly less than M (treat duplicate Ms as one maximum). If no value is smaller than M (all elements equal), print M.

Constraints: At least length two ints; duplicates allowed.

Sample Input:

5

3 9 9 1 4

Expected Output: 4

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

1 2

Expected output:

1

Case 2 (public)

Input:

4

4 4 4 4

Expected output:

4

Case 3 (public)

Input:

6

10 20 8 15 9 22

Expected output:

20

Case 4 (public)

Input:

3

100 75 90

Expected output:

90

Case 5 (public)

Input:

4

-2 -10 -10 -100

Expected output:

-10

89. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q4: Add Two Numbers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)**Input:**

10 5

Expected output:

15

Case 3 (public)**Input:**

-3 -4

Expected output:

-7

90. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q5: Simple Interest

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Simple Interest. Read input from stdin and print exact output.

Input Format: Three lines: principal P, rate R, time T (all integers).

Output Format: Print simple interest as integer.

Constraints: Three integers Principal Rate Time whole numbers.

Sample Input:

1000

5

2

Expected Output: 100

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

500

10

1

Expected output:

50

Case 2 (public)**Input:**

1500

8

2

Expected output:

240

Case 3 (public)**Input:**

9000

0

5

Expected output:

0

91. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q6: String Length

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

logical, Assignment'.

Problem style: String Length Manual. Read input from stdin and print exact output.

Input Format: Single line string (ASCII).

Output Format: Print length without using strlen.

Constraints: Non-empty string without embedded newline simplifying scanner.

Sample Input: coding

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

abcd

Expected output:

4

Case 2 (public)

Input:

race

Expected output:

4

Case 3 (public)

Input:

z

Expected output:

1

92. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q7: Calculator Switch Menu

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Calculator Switch Menu. Read input from stdin and print exact output.

Input Format: Line 1: op. Line 2: a. Line 3: b. Integer division for '/'.

Output Format: Print integer result.

Constraints: Operator letter then two operands lines.

Sample Input:

+

4

11

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

+

10

5

Expected output:

15

Case 2 (public)

Input:

/

49

7

Expected output:

7

Case 3 (public)

Input:

*

6

7

Expected output:

42

93. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q8: Strong Number Check

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Strong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Yes if sum of factorials of digits equals n, else No.

Constraints: Integer moderate digit factorial property.

Sample Input: 145

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

Yes

Case 2 (public)

Input:

2

Expected output:

Yes

Case 3 (public)

Input:

146

Expected output:

No

94. Phase 2 — Logic Building & Flow Control — Arithmetic, Relational, Logical, Assignment — Q9: Print Multiplication Table

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Arithmetic, Relational, Logical, Assignment'.

Problem style: Print Multiplication Table. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print first 10 multiples of n separated by spaces.

Constraints: One positive integer controlling the factor.

Sample Input: 5

Expected Output: 5 10 15 20 25 30 35 40 45 50

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1 2 3 4 5 6 7 8 9 10

Case 2 (public)

Input:

7

Expected output:

7 14 21 28 35 42 49 56 63 70

Case 3 (public)

Input:

3

Expected output:

3 6 9 12 15 18 21 24 27 30

95. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q10: Student Reco

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Student Record System. Read input from stdin and print exact output.

Input Format: n followed by n lines: name age.

Output Format: Print all student records line by line.

Constraints: Count then lines formatted name + integer age.

Sample Input:

2

Alice 20

Bob 22

Expected Output (sample):

Alice 20

Bob 22

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Tom 20

Sam 19

Expected output:

Tom 20

Sam 19

Case 2 (public)

Input:

1

Eve 25

Expected output:

Eve 25

Case 3 (public)

Input:

3

A 1

B 2

C 3

Expected output:

A 1

B 2

96. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q11: Sum Using Recursion

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Sum Using Recursion. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print sum of numbers from 1 to n using recursion.

Constraints: One positive integer n.

Sample Input: 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

3

Case 2 (public)

Input:

10

Expected output:

55

Case 3 (public)

Input:

7

Expected output:

28

97. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q12: Swap Using Pointers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Swap Using Pointers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped numbers using pointer logic.

Constraints: Two integers as usual swap problem.

Sample Input: 4 9

Expected Output: 9 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

-2 5

Expected output:

5 -2

Case 2 (public)

Input:

8 3

Expected output:

3 8

Case 3 (public)

Input:

0 7

Expected output:

7 0

98. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q13: Symmetric Matrix

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Symmetric Matrix Check. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Print Yes if symmetric ($A[i][j] == A[j][i]$), else No.

Constraints: 3×3 ints.

Sample Input:

1 2 3

2 4 5

3 5 6

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 0 0

0 1 0

0 0 1

Expected output:

Yes

Case 2 (public)

Input:

1 2 3

0 1 4

5 6 0

Expected output:

No

Case 3 (public)

Input:

1 2 3

2 4 5

3 5 6

Expected output:

Yes

99. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q14: Print Multiplication Table

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Print Multiplication Table. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print first 10 multiples of n separated by spaces.

Constraints: One positive integer controlling the factor.

Sample Input: 5

Expected Output: 5 10 15 20 25 30 35 40 45 50

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1 2 3 4 5 6 7 8 9 10

Case 2 (public)

Input:

7

Expected output:

7 14 21 28 35 42 49 56 63 70

Case 3 (public)

Input:

3

Expected output:

3 6 9 12 15 18 21 24 27 30

100. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q15: Prime Check

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Prime Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print exactly Prime or Not (capital P / capital N as shown).

Constraints: Any integer: $n < 2$ and negatives are Not; primes are ≥ 2 with no nontrivial divisor.

Sample Input: 7

Expected Output: Prime

Judge test case count: 6

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

Prime

Case 2 (public)

Input:

9

Expected output:

Not

Case 3 (public)

Input:

17

Expected output:

Prime

Case 4 (public)

Input:

1

Expected output:

Not

Case 5 (public)

Input:

0

Expected output:

Not

Case 6 (public)

Input:

-11

Expected output:

Not

101. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q16: Add Two Numbers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Add Two Numbers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print a + b.

Constraints: The user enters two ordinary integers.

Sample Input: 7 3

Expected Output: 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

4

Case 2 (public)

Input:

10 5

Expected output:

15

Case 3 (public)

Input:

-3 -4

Expected output:

-7

102. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q17: Read Until Zero

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Read Until Zero Sum. Read input from stdin and print exact output.

Input Format: One integer per line; last line is 0 to stop reading.

Output Format: Print sum of all integers before the line containing 0.

Constraints: User enters positive steps line-by-line stopping at sentinel line reading zero.

Sample Input:

2

12

18

20

0

Expected Output: 52

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1
7
9
17
0

Expected output:

34

Case 2 (public)

Input:

3
0

Expected output:

3

Case 3 (public)

Input:

10
20
5
0

Expected output:

35

103. Phase 2 — Logic Building & Flow Control — Increment and Decrement operators — Q18: Break On Negative Sum

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Increment and Decrement operators'.

Problem style: Break On Negative Sum. Read input from stdin and print exact output.

Input Format: First line count n, then n integers one per line (last may be negative).

Output Format: Print sum of values before first negative integer.

Constraints: Count prefixed then sequence entries until negative sentinel occurs.

Sample Input:

3
5
8
-2

Expected Output: 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4
1
2
3
-9

Expected output:

6

Case 2 (public)

Input:

1
-5

Expected output:

0

Case 3 (public)**Input:**

5
2
2
2
2
-1

Expected output:

8

104. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q19: Write Data To File

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Write Data To File. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Echo the same text (simulating file write/read).

Constraints: Single textual line echoed (simulates file write).

Sample Input: hello

Expected Output: hello

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

abc

Expected output:

abc

Case 2 (public)**Input:**

123

Expected output:

123

Case 3 (public)**Input:**

file data

Expected output:

file data

105. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q20: Access Array Using Pointer

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Access Array Using Pointer. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of array elements using pointer traversal.

Constraints: Length plus list of ints.

Sample Input:

4
2 4 6 8

Expected Output: 20

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

9

Expected output:

9

Case 2 (public)

Input:

5

1 2 3 4 5

Expected output:

15

Case 3 (public)

Input:

2

-1 4

Expected output:

3

106. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q21: Armstrong Number Check

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Armstrong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Let k be the number of decimal digits of n. n is an Armstrong (narcissistic) number if the sum of (each digit raised to the k-th power) equals n. Print Yes or No. Single-digit n always satisfies this (k=1), so every 1–9 is Yes.

Constraints: Generalized Armstrong: with k = digit count, sum of (each digit^k) equals n.

Sample Input: 153

Expected Output: Yes

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

Yes

Case 2 (public)

Input:

153

Expected output:

Yes

Case 3 (public)

Input:

9474

Expected output:

Yes

Case 4 (public)

Input:

200

Expected output:

No

Case 5 (public)

Input:

123

Expected output:

No

107. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q22: Array Reverse Print

Print

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Array Reverse Print. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print elements in reverse order, space-separated.

Constraints: `n` then list length n ints.

Sample Input:

4

1 2 3 4

Expected Output: 4 3 2 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

42

Expected output:

42

Case 2 (public)

Input:

2

0 5

Expected output:

5 0

Case 3 (public)

Input:

5

1 1 2 3 5

Expected output:

5 3 2 1 1

108. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q23: Check Even/Odd

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Check Even/Odd. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Even if n is even, else Odd.

Constraints: One integer (can be negative or zero).

Sample Input: 8

Expected Output: Even

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Even

Case 2 (public)

Input:

-3

Expected output:

Odd

Case 3 (public)

Input:

14

Expected output:

Even

109. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q24: Find Largest of 3 Numbers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Find Largest Of 3 Numbers. Read input from stdin and print exact output.

Input Format: Three integers a, b, c.

Output Format: Print the largest value.

Constraints: Three integers separated on one line (or consecutive reads).

Sample Input: 9 2 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3

Expected output:

3

Case 2 (public)

Input:

5 5 1

Expected output:

5

Case 3 (public)

Input:

7 4 7

Expected output:

7

110. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q25: Grade Calculator

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Grade Calculator. Read input from stdin and print exact output.

Input Format: One integer marks.

Output Format: Print grade: A (≥ 90), B (≥ 80), C (≥ 70), D (≥ 60), F (< 60).

Constraints: One mark from 0 to 100.

Sample Input: 82

Expected Output: B

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

74

Expected output:

C

Case 2 (public)

Input:

60

Expected output:

D

Case 3 (public)

Input:

48

Expected output:

F

111. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q26: Day Of Week

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Day Of Week Switch. Read input from stdin and print exact output.

Input Format: One integer.

Output Format: Print abbreviated weekday exactly: Mon Tue Wed Thu Fri Sat Sun

Constraints: Integer 1..7 referencing weekday numbering from handout.

Sample Input: 3

Expected Output: Wed

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4

Expected output:

Thu

Case 2 (public)

Input:

6

Expected output:

Sat

Case 3 (public)

Input:

2

Expected output:

Tue

112. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q27: Prime Check

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Prime Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print exactly Prime or Not (capital P / capital N as shown).

Constraints: Any integer: $n < 2$ and negatives are Not; primes are ≥ 2 with no nontrivial divisor.

Sample Input: 7

Expected Output: Prime

Judge test case count: 6

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

Prime

Case 2 (public)

Input:

9

Expected output:

Not

Case 3 (public)

Input:

17

Expected output:

Prime

Case 4 (public)

Input:

1

Expected output:

Not

Case 5 (public)

Input:

0

Expected output:

Not

Case 6 (public)

Input:

-11

Expected output:

Not

113. Phase 2 — Logic Building & Flow Control — Conditional Statements (if, else, switch) — Q28: Leap Year

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Conditional Statements (if, else, switch)'.

Problem style: Leap Year. Read input from stdin and print exact output.

Input Format: One integer year.

Output Format: Print Leap or Not

Constraints: One calendar year typed.

Sample Input: 2024

Expected Output: Leap

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2000

Expected output:

Leap

Case 2 (public)

Input:

1900

Expected output:

Not

Case 3 (public)

Input:

1996

Expected output:

Leap

114. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q29: Break On Negative Sum

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Break On Negative Sum. Read input from stdin and print exact output.

Input Format: First line count n, then n integers one per line (last may be negative).

Output Format: Print sum of values before first negative integer.

Constraints: Count prefixed then sequence entries until negative sentinel occurs.

Sample Input:

3

5

8

-2

Expected Output: 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4

1

2

3

-9

Expected output:

6

Case 2 (public)

Input:

1

-5

Expected output:

0

Case 3 (public)

Input:

5

2

2

2

2

-1

Expected output:

8

115. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q30: Bubble Sort Array

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Bubble Sort Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Print sorted ascending, space-separated (bubble sort / any stable $O(n^2)$ ok).

Constraints: `n` moderate sortable ints.

Sample Input:

4

4 1 3 2

Expected Output: 1 2 3 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

2 1

Expected output:

1 2

Case 2 (public)

Input:

5

5 4 3 2 1

Expected output:

1 2 3 4 5

Case 3 (public)

Input:

1

7

Expected output:

7

116. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q31: Calculator Switch Me

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Calculator Switch Menu. Read input from stdin and print exact output.

Input Format: Line 1: op. Line 2: a. Line 3: b. Integer division for '/'.

Output Format: Print integer result.

Constraints: Operator letter then two operands lines.

Sample Input:

+

4

11

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

+
10
5

Expected output:

15

Case 2 (public)

Input:

/
49
7

Expected output:

7

Case 3 (public)

Input:

*
6
7

Expected output:

42

117. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q32: Check Palindrome

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Check Palindrome. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print Yes if palindrome, else No.

Constraints: One string with no gaps.

Sample Input: madam

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

Yes

Case 2 (public)

Input:

abba

Expected output:

Yes

Case 3 (public)

Input:

abc

Expected output:

No

118. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q33: Print Multiplication Table

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Print Multiplication Table. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print first 10 multiples of n separated by spaces.

Constraints: One positive integer controlling the factor.

Sample Input: 5

Expected Output: 5 10 15 20 25 30 35 40 45 50

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1 2 3 4 5 6 7 8 9 10

Case 2 (public)

Input:

7

Expected output:

7 14 21 28 35 42 49 56 63 70

Case 3 (public)

Input:

3

Expected output:

3 6 9 12 15 18 21 24 27 30

119. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q34: Read Until Zero Sum

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Read Until Zero Sum. Read input from stdin and print exact output.

Input Format: One integer per line; last line is 0 to stop reading.

Output Format: Print sum of all integers before the line containing 0.

Constraints: User enters positive steps line-by-line stopping at sentinel line reading zero.

Sample Input:

2

12

18

20

0

Expected Output: 52

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

9

17

0

Expected output:

34

Case 2 (public)

Input:

3

0

Expected output:

3

Case 3 (public)

Input:

10

20

5

0

Expected output:

35

120. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q35: Find Largest Of 3 Numbers

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.
Problem style: Find Largest Of 3 Numbers. Read input from stdin and print exact output.

Input Format: Three integers a, b, c.

Output Format: Print the largest value.

Constraints: Three integers separated on one line (or consecutive reads).

Sample Input: 9 2 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3

Expected output:

3

Case 2 (public)

Input:

5 5 1

Expected output:

5

Case 3 (public)

Input:

7 4 7

Expected output:

7

121. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q36: Fibonacci Series

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.
Problem style: Fibonacci Series. Read input from stdin and print exact output.

Input Format: One integer n (number of terms).

Output Format: Print first n Fibonacci numbers separated by spaces.

Constraints: Count of terms to emit.

Sample Input: 7

Expected Output: 0 1 1 2 3 5 8

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

0 1

Case 2 (public)

Input:

3

Expected output:

0 1 1

Case 3 (public)

Input:

6

Expected output:

0 1 1 2 3 5

122. Phase 2 — Logic Building & Flow Control — Loops (for, while, do-while) — Q37: Factorial Of Number

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Loops (for, while, do-while)'.

Problem style: Factorial Of Number. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print n!.

Constraints: Small non-negative integer n (factorial grows fast).

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

6

Expected output:

720

Case 3 (public)

Input:

3

Expected output:

6

123. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q38: Leap Year

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Leap Year. Read input from stdin and print exact output.

Input Format: One integer year.

Output Format: Print Leap or Not

Constraints: One calendar year typed.

Sample Input: 2024

Expected Output: Leap

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)**Input:**

2000

Expected output:

Leap

Case 2 (public)**Input:**

1900

Expected output:

Not

Case 3 (public)**Input:**

1996

Expected output:

Leap

124. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q39: Concatenate Without Strcat

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Concatenate Without Strcat. Read input from stdin and print exact output.

Input Format: Line 1 string A; line 2 string B.

Output Format: Print A+B exactly.

Constraints: Two textual tokens sequentially lines.

Sample Input:

code

lab

Expected Output: codelab

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

x

yz

Expected output:

xyz

Case 2 (public)**Input:**

tree

House

Expected output:

treeHouse

Case 3 (public)**Input:**

a

b

Expected output:

ab

125. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q40: Copy File

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, c

ontinue, goto)'.

Problem style: Copy File. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Print the same text (simulating copy output).

Constraints: One line echoed as duplicated output.

Sample Input: copy this

Expected Output: copy this

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

mno

Expected output:

mno

Case 2 (public)

Input:

data

Expected output:

data

Case 3 (public)

Input:

done

Expected output:

done

126. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q41: Count Above Average

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Count Above Average. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Count elements strictly greater than floor(average); average = floor(sum/n).

Constraints: Array ints general distribution.

Sample Input:

5

2 2 2 2 8

Expected Output: 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3

10 10 10

Expected output:

0

Case 2 (public)

Input:

4

9 10 11 100

Expected output:

1

Case 3 (public)

Input:

2

5 15

Expected output:

1

127. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q42: Count Vowels

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Count Vowels. Read input from stdin and print exact output.

Input Format: One lowercase/uppercase string.

Output Format: Print vowel count (a, e, i, o, u).

Constraints: One line mix of uppercase/lowercase letters.

Sample Input: education

Expected Output: 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

AEIOU

Expected output:

5

Case 2 (public)

Input:

programming

Expected output:

3

Case 3 (public)

Input:

bcd

Expected output:

0

128. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q43: Break On Negative Sum

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Break On Negative Sum. Read input from stdin and print exact output.

Input Format: First line count n, then n integers one per line (last may be negative).

Output Format: Print sum of values before first negative integer.

Constraints: Count prefixed then sequence entries until negative sentinel occurs.

Sample Input:

3

5

8

-2

Expected Output: 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4

1
2
3
-9

Expected output:

6

Case 2 (public)

Input:

1
-5

Expected output:

0

Case 3 (public)

Input:

5
2
2
2
2
-1

Expected output:

8

129. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q44: Read Until Zero Sum

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Read Until Zero Sum. Read input from stdin and print exact output.

Input Format: One integer per line; last line is 0 to stop reading.

Output Format: Print sum of all integers before the line containing 0.

Constraints: User enters positive steps line-by-line stopping at sentinel line reading zero.

Sample Input:

2
12
18
20
0

Expected Output: 52

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1
7
9
17
0

Expected output:

34

Case 2 (public)

Input:

3
0

Expected output:

3

Case 3 (public)

Input:

10

20

5

0

Expected output:

35

130. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q45: Print Multiplication Table

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Print Multiplication Table. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print first 10 multiples of n separated by spaces.

Constraints: One positive integer controlling the factor.

Sample Input: 5

Expected Output: 5 10 15 20 25 30 35 40 45 50

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1 2 3 4 5 6 7 8 9 10

Case 2 (public)

Input:

7

Expected output:

7 14 21 28 35 42 49 56 63 70

Case 3 (public)

Input:

3

Expected output:

3 6 9 12 15 18 21 24 27 30

131. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q46: Check Even/Odd

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Check Even/Odd. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Even if n is even, else Odd.

Constraints: One integer (can be negative or zero).

Sample Input: 8

Expected Output: Even

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Even

Case 2 (public)

Input:

-3

Expected output:

Odd

Case 3 (public)

Input:

14

Expected output:

Even

132. Phase 2 — Logic Building & Flow Control — Jump statements (break, continue, goto) — Q47: Simple Interest

Module: Phase 2 — Logic Building & Flow Control

Difficulty: easy

Description: Solve a phase 2 — logic building & flow control coding task focused on 'Jump statements (break, continue, goto)'.

Problem style: Simple Interest. Read input from stdin and print exact output.

Input Format: Three lines: principal P, rate R, time T (all integers).

Output Format: Print simple interest as integer.

Constraints: Three integers Principal Rate Time whole numbers.

Sample Input:

1000

5

2

Expected Output: 100

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

500

10

1

Expected output:

50

Case 2 (public)

Input:

1500

8

2

Expected output:

240

Case 3 (public)

Input:

9000

0

5

Expected output:

0

133. Phase 3 — Functions — Function declaration, definition, and call — Q1: Print Primes In Range

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Print Primes In Range. Read input from stdin and print exact output.

Input Format: One line with two integers lo and hi separated by whitespace.

Output Format: Print all primes in [lo, hi] inclusive, space-separated, ascending.

Constraints: Two bounds lo and hi on one line (space-separated), inclusive range.

Sample Input: 5 15

Expected Output: 5 7 11 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

2

Case 2 (public)

Input:

14 18

Expected output:

17

Case 3 (public)

Input:

20 30

Expected output:

23 29

134. Phase 3 — Functions — Function declaration, definition, and call — Q2: Read Name And Age Greeting

Module: Phase 3 — Functions

Difficulty: easy

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Read Name And Age Greeting. Read input from stdin and print exact output.

Input Format: Line 1: name string. Line 2: integer age.

Output Format: Print one greeting sentence exactly.

Constraints: Person gives one word nickname, then a normal human age.

Sample Input:

Alex

21

Expected Output: Hello Alex, you are 21 years old.

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Amy

17

Expected output:

Hello Amy, you are 17 years old.

Case 2 (public)

Input:

Leo

0

Expected output:

Hello Leo, you are 0 years old.

Case 3 (public)

Input:

Zoe

100

Expected output:

Hello Zoe, you are 100 years old.

135. Phase 3 — Functions — Function declaration, definition, and call — Q3: Fibonacci Series

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Fibonacci Series. Read input from stdin and print exact output.

Input Format: One integer n (number of terms).

Output Format: Print first n Fibonacci numbers separated by spaces.

Constraints: Count of terms to emit.

Sample Input: 7

Expected Output: 0 1 1 2 3 5 8

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

0 1

Case 2 (public)

Input:

3

Expected output:

0 1 1

Case 3 (public)

Input:

6

Expected output:

0 1 1 2 3 5

136. Phase 3 — Functions — Function declaration, definition, and call — Q4: Count Vowels Consonants Spaces

Module: Phase 3 — Functions

Difficulty: easy

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Count Vowels Consonants Spaces. Read input from stdin and print exact output.

Input Format: One line: sentence.

Output Format: Print three integers: vowels (aeiou AEIOU), consonants (other letters), spaces.

Constraints: One sentence letters spaces only simplifying classification.

Sample Input: Hi there

Expected Output: 3 4 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

ABC

Expected output:

1 2 0

Case 2 (public)

Input:

a b c

Expected output:

1 2 2

Case 3 (public)

Input:

rhythm

Expected output:

0 6 0

137. Phase 3 — Functions — Function declaration, definition, and call — Q5: Factorial Of Number

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Factorial Of Number. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print n!.

Constraints: Small non-negative integer n (factorial grows fast).

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1

Expected output:

1

Case 2 (public)**Input:**

6

Expected output:

720

Case 3 (public)**Input:**

3

Expected output:

6

138. Phase 3 — Functions — Function declaration, definition, and call — Q6: Employee Details

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Employee Details. Read input from stdin and print exact output.

Input Format: n followed by n lines: id name.

Output Format: Print employee details as id name per line.

Constraints: Count then lines with id integer and name.

Sample Input:

2
101 Alice
102 Bob

Expected Output (sample):

101 Alice
102 Bob

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1

301 Mia

Expected output:

301 Mia

Case 2 (public)

Input:

2

11 Raj

12 Dee

Expected output:

11 Raj

12 Dee

Case 3 (public)

Input:

3

1 X

2 Y

3 Z

Expected output:

1 X

2 Y

3 Z

139. Phase 3 — Functions — Function declaration, definition, and call — Q7: Is Even Function Loop

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Is Even Function Loop. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print all even numbers from 2 to n inclusive, space-separated.

Constraints: Upper bound counting evens inclusively downward.

Sample Input: 10

Expected Output: 2 4 6 8 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

(empty)

Case 2 (public)

Input:

4

Expected output:

2 4

Case 3 (public)

Input:

12

Expected output:

2 4 6 8 10 12

140. Phase 3 — Functions — Function declaration, definition, and call — Q8: Equilateral Star Pattern

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Equilateral Star Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows in the centered star triangle).

Output Format: For each row r from 0 to n-1 (top to bottom): print exactly (n-1-r) spaces, then (r+1) stars wi

th a single space between consecutive stars, and no space after the last star on that line. This centers each row so column alignment matches the sample for every test case.

Constraints: Same spacing rule for every row: row r uses $(n-1-r)$ spaces then $r+1$ stars with single spaces between stars; no trailing spaces.

Sample Input: 4

Expected Output (sample):

```
*
* *
* * *
* * * *
```

Judge test case count: 4

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

```
*
```

Case 2 (public)

Input:

2

Expected output:

```
*
* *
```

Case 3 (public)

Input:

3

Expected output:

```
*
* *
* * *
```

Case 4 (public)

Input:

4

Expected output:

```
*
* *
* * *
* * * *
```

141. Phase 3 — Functions — Function declaration, definition, and call — Q9: Factorial Using Function

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Factorial Using Function. Read input from stdin and print exact output.

Input Format: One integer n .

Output Format: Print factorial of n using a function.

Constraints: Same as factorial puzzle but enforce your own helper.

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

1

Case 2 (public)

Input:

1

Expected output:

1

Case 3 (public)

Input:

6

Expected output:

720

142. Phase 3 — Functions — Function declaration, definition, and call — Q10: Sum Using Recursion

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Sum Using Recursion. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print sum of numbers from 1 to n using recursion.

Constraints: One positive integer n.

Sample Input: 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

3

Case 2 (public)

Input:

10

Expected output:

55

Case 3 (public)

Input:

7

Expected output:

28

143. Phase 3 — Functions — Function declaration, definition, and call — Q11: Gcd Euclidean

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Gcd Euclidean. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print GCD(a,b).

Constraints: Two positive integers on one line (space-separated), same as typical scanf pair input.

Sample Input: 48 18

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

7 13

Expected output:

1

Case 2 (public)

Input:

54 24

Expected output:

6

Case 3 (public)

Input:

17 17

Expected output:

17

144. Phase 3 — Functions — Function declaration, definition, and call — Q12: Lcm Using Gcd

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Function declaration, definition, and call'.

Problem style: Lcm Using Gcd. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print LCM(a,b) using $LCM = a*b/GCD(a,b)$ with integer arithmetic.

Constraints: Two integers on one line (space-separated).

Sample Input: 4 6

Expected Output: 12

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3 9

Expected output:

9

Case 2 (public)

Input:

8 12

Expected output:

24

Case 3 (public)

Input:

11 13

Expected output:

143

145. Phase 3 — Functions — Call by value and call by reference — Q13: Student Record Variables

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Student Record Variables. Read input from stdin and print exact output.

Input Format: Four lines: name, age, grade letter, tuition fees.

Output Format: Print four fields separated by spaces: name age grade fees.

Constraints: Four separate answers: word name, numeric age, one letter grade, tuition digits.

Sample Input:

Mia

19

B

72000

Expected Output: Mia 19 B 72000

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

Raj
22
C
61000

Expected output:

Raj 22 C 61000

Case 2 (public)

Input:

Zoe
18
F
8000

Expected output:

Zoe 18 F 8000

Case 3 (public)

Input:

Leo
30
D
41234

Expected output:

Leo 30 D 41234

146. Phase 3 — Functions — Call by value and call by reference — Q14: Fibonacci Series

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Fibonacci Series. Read input from stdin and print exact output.

Input Format: One integer n (number of terms).

Output Format: Print first n Fibonacci numbers separated by spaces.

Constraints: Count of terms to emit.

Sample Input: 7

Expected Output: 0 1 1 2 3 5 8

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

0 1

Case 2 (public)

Input:

3

Expected output:

0 1 1

Case 3 (public)

Input:

6

Expected output:

0 1 1 2 3 5

147. Phase 3 — Functions — Call by value and call by reference — Q15: Find Sum Of Array

Module: Phase 3 — Functions

Difficulty: easy

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Find Sum Of Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of elements.

Constraints: First n, then exactly n integers on the next input line.

Sample Input:

```
5
1 2 3 4 5
```

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1
10
```

Expected output:

```
10
```

Case 2 (public)

Input:

```
4
0 0 0 0
```

Expected output:

```
0
```

Case 3 (public)

Input:

```
3
7 8 9
```

Expected output:

```
24
```

148. Phase 3 — Functions — Call by value and call by reference — Q16: Floyd Triangle Pattern

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Floyd Triangle Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows).

Output Format: Floyd triangle: row i has i consecutive integers, numbers separated by a single space; rows separated by newline. Each line ends immediately after the last digit—no trailing spaces.

Constraints: Height rows grows consecutive integers pattern.

Sample Input: 3

Expected Output (sample):

```
1
2 3
4 5 6
```

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1
```

Expected output:

```
1
```

Case 2 (public)

Input:

4

Expected output:

1

2 3

4 5 6

7 8 9 10

Case 3 (public)

Input:

3

Expected output:

1

2 3

4 5 6

149. Phase 3 — Functions — Call by value and call by reference — Q17: Access Array Using Pointer

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Access Array Using Pointer. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of array elements using pointer traversal.

Constraints: Length plus list of ints.

Sample Input:

4

2 4 6 8

Expected Output: 20

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

9

Expected output:

9

Case 2 (public)

Input:

5

1 2 3 4 5

Expected output:

15

Case 3 (public)

Input:

2

-1 4

Expected output:

3

150. Phase 3 — Functions — Call by value and call by reference — Q18: Factorial Using Function

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Factorial Using Function. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print factorial of n using a function.

Constraints: Same as factorial puzzle but enforce your own helper.

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

1

Case 2 (public)

Input:

1

Expected output:

1

Case 3 (public)

Input:

6

Expected output:

720

151. Phase 3 — Functions — Call by value and call by reference — Q19: Gcd Euclidean

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Gcd Euclidean. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print GCD(a,b).

Constraints: Two positive integers on one line (space-separated), same as typical scanf pair input.

Sample Input: 48 18

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

7 13

Expected output:

1

Case 2 (public)

Input:

54 24

Expected output:

6

Case 3 (public)

Input:

17 17

Expected output:

17

152. Phase 3 — Functions — Call by value and call by reference — Q20: Lcm Using Gcd

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Lcm Using Gcd. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print LCM(a,b) using $LCM = a*b/GCD(a,b)$ with integer arithmetic.

Constraints: Two integers on one line (space-separated).

Sample Input: 4 6

Expected Output: 12

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3 9

Expected output:

9

Case 2 (public)

Input:

8 12

Expected output:

24

Case 3 (public)

Input:

11 13

Expected output:

143

153. Phase 3 — Functions — Call by value and call by reference — Q21: Sum Using Recursion

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Sum Using Recursion. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print sum of numbers from 1 to n using recursion.

Constraints: One positive integer n.

Sample Input: 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

3

Case 2 (public)

Input:

10

Expected output:

55

Case 3 (public)

Input:

7

Expected output:

28

154. Phase 3 — Functions — Call by value and call by reference — Q22: Swap Using Pointers

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Swap Using Pointers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped numbers using pointer logic.

Constraints: Two integers as usual swap problem.

Sample Input: 4 9

Expected Output: 9 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

-2 5

Expected output:

5 -2

Case 2 (public)

Input:

8 3

Expected output:

3 8

Case 3 (public)

Input:

0 7

Expected output:

7 0

155. Phase 3 — Functions — Call by value and call by reference — Q23: Armstrong Number Check

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Call by value and call by reference'.

Problem style: Armstrong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Let k be the number of decimal digits of n. n is an Armstrong (narcissistic) number if the sum of (each digit raised to the k-th power) equals n. Print Yes or No. Single-digit n always satisfies this (k=1), so every 1–9 is Yes.

Constraints: Generalized Armstrong: with k = digit count, sum of (each digit^k) equals n.

Sample Input: 153

Expected Output: Yes

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

Yes

Case 2 (public)

Input:

153

Expected output:

Yes

Case 3 (public)

Input:

9474

Expected output:

Yes

Case 4 (public)

Input:

200

Expected output:

No

Case 5 (public)

Input:

123

Expected output:

No

156. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q24: Day Of Week Switch

Module: Phase 3 — Functions

Difficulty: easy

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial, gcd, lcm'.

Problem style: Day Of Week Switch. Read input from stdin and print exact output.

Input Format: One integer.

Output Format: Print abbreviated weekday exactly: Mon Tue Wed Thu Fri Sat Sun

Constraints: Integer 1..7 referencing weekday numbering from handout.

Sample Input: 3

Expected Output: Wed

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4

Expected output:

Thu

Case 2 (public)

Input:

6

Expected output:

Sat

Case 3 (public)

Input:

2

Expected output:

Tue

157. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q25: Is Even Function

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial, gcd, lcm'.

Problem style: Is Even Function Loop. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print all even numbers from 2 to n inclusive, space-separated.

Constraints: Upper bound counting evens inclusively downward.

Sample Input: 10

Expected Output: 2 4 6 8 10

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

(empty)

Case 2 (public)

Input:

4

Expected output:

2 4

Case 3 (public)

Input:

12

Expected output:

2 4 6 8 10 12

158. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q26: Largest Element

Module: Phase 3 — Functions

Difficulty: easy

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial, gcd, lcm'.

Problem style: Largest Element. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print largest array element.

Constraints: n then n integers.

Sample Input:

5

1 2 9 4 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

Expected output:

7

Case 2 (public)

Input:

4

2 2 2 2

Expected output:

2

Case 3 (public)

Input:

5

8 1 3 6 4

Expected output:

8

159. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q27: Print Primes In Range

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial, gcd, lcm'.

Problem style: Print Primes In Range. Read input from stdin and print exact output.

Input Format: One line with two integers lo and hi separated by whitespace.

Output Format: Print all primes in [lo, hi] inclusive, space-separated, ascending.

Constraints: Two bounds lo and hi on one line (space-separated), inclusive range.

Sample Input: 5 15

Expected Output: 5 7 11 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

2

Case 2 (public)

Input:

14 18

Expected output:

17

Case 3 (public)

Input:

20 30

Expected output:

23 29

160. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q28: Armstrong Number Check

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial, gcd, lcm'.

Problem style: Armstrong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Let k be the number of decimal digits of n. n is an Armstrong (narcissistic) number if the sum of (each digit raised to the k-th power) equals n. Print Yes or No. Single-digit n always satisfies this (k=1), so every 1–9 is Yes.

Constraints: Generalized Armstrong: with k = digit count, sum of (each digit^k) equals n.

Sample Input: 153

Expected Output: Yes

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

Yes

Case 2 (public)

Input:

153

Expected output:

Yes

Case 3 (public)

Input:

9474

Expected output:

Yes

Case 4 (public)

Input:

200

Expected output:

No

Case 5 (public)

Input:

123

Expected output:

No

161. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q29: Gcd Euclidean

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial , gcd, lcm'.

Problem style: Gcd Euclidean. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print GCD(a,b).

Constraints: Two positive integers on one line (space-separated), same as typical scanf pair input.

Sample Input: 48 18

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

7 13

Expected output:

1

Case 2 (public)

Input:

54 24

Expected output:

6

Case 3 (public)

Input:

17 17

Expected output:

17

162. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q30: Lcm Using Gcd

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial , gcd, lcm'.

Problem style: Lcm Using Gcd. Read input from stdin and print exact output.

Input Format: One line with two integers a and b separated by whitespace.

Output Format: Print LCM(a,b) using $LCM = a*b/GCD(a,b)$ with integer arithmetic.

Constraints: Two integers on one line (space-separated).

Sample Input: 4 6

Expected Output: 12

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3 9

Expected output:

9

Case 2 (public)**Input:**

8 12

Expected output:

24

Case 3 (public)**Input:**

11 13

Expected output:

143

163. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q31: Prime Check

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial , gcd, lcm'.

Problem style: Prime Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print exactly Prime or Not (capital P / capital N as shown).

Constraints: Any integer: $n < 2$ and negatives are Not; primes are ≥ 2 with no nontrivial divisor.

Sample Input: 7

Expected Output: Prime

Judge test case count: 6

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

2

Expected output:

Prime

Case 2 (public)**Input:**

9

Expected output:

Not

Case 3 (public)**Input:**

17

Expected output:

Prime

Case 4 (public)**Input:**

1

Expected output:

Not

Case 5 (public)**Input:**

0

Expected output:

Not

Case 6 (public)**Input:**

-11

Expected output:

Not

164. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q32: Factorial Of Number

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial , gcd, lcm'.

Problem style: Factorial Of Number. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print n!.

Constraints: Small non-negative integer n (factorial grows fast).

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

6

Expected output:

720

Case 3 (public)

Input:

3

Expected output:

6

165. Phase 3 — Functions — Using functions for primes, powers, factorial, gcd, lcm — Q33: Floyd Triangle Pattern

Module: Phase 3 — Functions

Difficulty: medium

Description: Solve a phase 3 — functions coding task focused on 'Using functions for primes, powers, factorial , gcd, lcm'.

Problem style: Floyd Triangle Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows).

Output Format: Floyd triangle: row i has i consecutive integers, numbers separated by a single space; rows separated by newline. Each line ends immediately after the last digit—no trailing spaces.

Constraints: Height rows grows consecutive integers pattern.

Sample Input: 3

Expected Output (sample):

1

2 3

4 5 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

4

Expected output:

```
1
2 3
4 5 6
7 8 9 10
```

Case 3 (public)

Input:

```
3
```

Expected output:

```
1
2 3
4 5 6
```

166. Phase 4 — Data Collections — One-dimensional arrays — Q1: Array Reverse Print

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Array Reverse Print. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print elements in reverse order, space-separated.

Constraints: `n` then list length n ints.

Sample Input:

```
4
1 2 3 4
```

Expected Output: 4 3 2 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1
42
```

Expected output:

```
42
```

Case 2 (public)

Input:

```
2
0 5
```

Expected output:

```
5 0
```

Case 3 (public)

Input:

```
5
1 1 2 3 5
```

Expected output:

```
5 3 2 1 1
```

167. Phase 4 — Data Collections — One-dimensional arrays — Q2: Print Simple Box Border

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Print Simple Box Border. Read input from stdin and print exact output.

Input Format: One integer n (side length of hollow square drawn with '*'). Middle uses spaces.

Output Format: Print exactly n rows: rows 1 and n are n stars; rows 2..n-1 are star, (n-2 spaces), star. Each line ends right after the last character—no trailing spaces.

Constraints: Border size decides rows/columns counts for hollow square artwork.

Sample Input: 3

Expected Output (sample):

* *

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

3

Expected output:

* *

Case 2 (public)

Input:

4

Expected output:

* *

* *

Case 3 (public)

Input:

5

Expected output:

* *

* *

* *

168. Phase 4 — Data Collections — One-dimensional arrays — Q3: Find Sum Of Array

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Find Sum Of Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of elements.

Constraints: First n, then exactly n integers on the next input line.

Sample Input:

5

1 2 3 4 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

10

Expected output:

10

Case 2 (public)

Input:

4

0 0 0 0

Expected output:

0

Case 3 (public)

Input:

3

7 8 9

Expected output:

24

169. Phase 4 — Data Collections — One-dimensional arrays — Q4: Largest Element

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Largest Element. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print largest array element.

Constraints: n then n integers.

Sample Input:

5

1 2 9 4 5

Expected Output: 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

Expected output:

7

Case 2 (public)

Input:

4

2 2 2 2

Expected output:

2

Case 3 (public)

Input:

5

8 1 3 6 4

Expected output:

8

170. Phase 4 — Data Collections — One-dimensional arrays — Q5: Matrix Addition

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Matrix Addition. Read input from stdin and print exact output.

Input Format: r c, then matrix A (r lines), then matrix B (r lines).

Output Format: Print A+B matrix row-wise.

Constraints: Dimensions plus two matrices stacked in stdin order.

Sample Input:

2 2

1 2

3 4

5 6

7 8

Expected Output (sample):

6 8

10 12

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 1

2

7

Expected output:

9

Case 2 (public)

Input:

2 2

1 1

1 1

2 2

2 2

Expected output:

3 3

3 3

Case 3 (public)

Input:

1 3

1 2 3

3 2 1

Expected output:

4 4 4

171. Phase 4 — Data Collections — One-dimensional arrays — Q6: Max Min Average Array

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Max Min Average Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print max min average as integers: $\text{average} = \text{floor}(\text{sum}/n)$.

Constraints: `n` then n ints manageable array sizes.

Sample Input:

4

2 8 4 6

Expected Output: 8 2 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

7

Expected output:

7 7 7

Case 2 (public)

Input:

2

1 2

Expected output:

2 1 1

Case 3 (public)

Input:

5

1 1 1 1 1

Expected output:

1 1 1

172. Phase 4 — Data Collections — One-dimensional arrays — Q7: Matrix Print 3X3

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Matrix Print 3X3. Read input from stdin and print exact output.

Input Format: Three lines, each with three integers.

Output Format: Print matrix row-wise with spaces between numbers per row.

Constraints: Exactly nine ints row major three lines typed.

Sample Input:

1 2 3

4 5 6

7 8 9

Expected Output (sample):

1 2 3

4 5 6

7 8 9

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9 8 7

6 5 4

3 2 1

Expected output:

9 8 7

6 5 4

3 2 1

Case 2 (public)

Input:

1 0 0

0 1 0

0 0 1

Expected output:

1 0 0

0 1 0

0 0 1

173. Phase 4 — Data Collections — One-dimensional arrays — Q8: Second Largest Array

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Second Largest Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Compute M = maximum element. If some array value is smaller than M, print the largest among values strictly less than M (treat duplicate Ms as one maximum). If no value is smaller than M (all elements equal to M), print M.

l), print M.

Constraints: At least length two ints; duplicates allowed.

Sample Input:

5
3 9 9 1 4

Expected Output: 4

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2
1 2

Expected output:

1

Case 2 (public)

Input:

4
4 4 4 4

Expected output:

4

Case 3 (public)

Input:

6
10 20 8 15 9 22

Expected output:

20

Case 4 (public)

Input:

3
100 75 90

Expected output:

90

Case 5 (public)

Input:

4
-2 -10 -10 -100

Expected output:

-10

174. Phase 4 — Data Collections — One-dimensional arrays — Q9: Matrix Row Column Sums

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Matrix Row Column Sums. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Line 1: row sums space-separated. Line 2: column sums space-separated.

Constraints: `3 × 3` numeric grid unchanged.

Sample Input:

1 2 3
4 5 6
7 8 9

Expected Output (sample):

6 15 24
12 15 18

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3
4 5 6
7 8 9

Expected output:

6 15 24
12 15 18

Case 2 (public)

Input:

0 0 1
0 1 0
1 0 0

Expected output:

1 1 1
1 1 1

175. Phase 4 — Data Collections — One-dimensional arrays — Q10: Linear Search Array

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Linear Search Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers. Line 3: target.

Output Format: Print 0-based index of first occurrence, or -1 if not found.

Constraints: Array listing plus target finder.

Sample Input:

5
1 4 7 9 2
7

Expected Output: 2

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

4
8 8 8 8
8

Expected output:

0

Case 2 (public)

Input:

2
1 2
3

Expected output:

-1

Case 3 (public)

Input:

1
42
42

Expected output:

0

176. Phase 4 — Data Collections — One-dimensional arrays — Q11: Count Above Average

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'One-dimensional arrays'.

Problem style: Count Above Average. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Count elements strictly greater than floor(average); average = floor(sum/n).

Constraints: Array ints general distribution.

Sample Input:

```
5
2 2 2 2 8
```

Expected Output: 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
3
10 10 10
```

Expected output:

```
0
```

Case 2 (public)

Input:

```
4
9 10 11 100
```

Expected output:

```
1
```

Case 3 (public)

Input:

```
2
5 15
```

Expected output:

```
1
```

177. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q12: Max Min Average Array

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Max Min Average Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print max min average as integers: average = floor(sum/n).

Constraints: `n` then n ints manageable array sizes.

Sample Input:

```
4
2 8 4 6
```

Expected Output: 8 2 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1
7
```

Expected output:

```
7 7 7
```

Case 2 (public)

Input:

2

1 2

Expected output:

2 1 1

Case 3 (public)

Input:

5

1 1 1 1 1

Expected output:

1 1 1

178. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q13: Palindrome Number

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Palindrome Number. Read input from stdin and print exact output.

Input Format: One integer n (no leading zeros except single 0).

Output Format: Print Yes if decimal digits read same forward/backward, else No.

Constraints: Integer decimal digit symmetry checking.

Sample Input: 121

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Yes

Case 2 (public)

Input:

1221

Expected output:

Yes

Case 3 (public)

Input:

10

Expected output:

No

179. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q14: Pascal Triangle Rows

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Pascal Triangle Rows. Read input from stdin and print exact output.

Input Format: One integer n = number of rows.

Output Format: Print Pascal triangle: row i has i integers, space-separated.

Constraints: `n` rows modest combinator growth.

Sample Input: 3

Expected Output (sample):

1

1 1

1 2 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

2

Expected output:

1

1 1

Case 3 (public)

Input:

5

Expected output:

1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

180. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q15: Pointer To Pointer

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Pointer To Pointer. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print value using pointer-to-pointer dereference.

Constraints: One integer echoed conceptually.

Sample Input: 7

Expected Output: 7

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

0

Case 2 (public)

Input:

15

Expected output:

15

Case 3 (public)

Input:

-8

Expected output:

-8

181. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q16: Matrix Print 3X3

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Matrix Print 3X3. Read input from stdin and print exact output.

Input Format: Three lines, each with three integers.

Output Format: Print matrix row-wise with spaces between numbers per row.

Constraints: Exactly nine ints row major three lines typed.

Sample Input:

```
1 2 3
4 5 6
7 8 9
```

Expected Output (sample):

```
1 2 3
4 5 6
7 8 9
```

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
9 8 7
6 5 4
3 2 1
```

Expected output:

```
9 8 7
6 5 4
3 2 1
```

Case 2 (public)

Input:

```
1 0 0
0 1 0
0 0 1
```

Expected output:

```
1 0 0
0 1 0
0 0 1
```

182. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q17: Matrix Row Column Sums

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Matrix Row Column Sums. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Line 1: row sums space-separated. Line 2: column sums space-separated.

Constraints: 3×3 numeric grid unchanged.

Sample Input:

```
1 2 3
4 5 6
7 8 9
```

Expected Output (sample):

```
6 15 24
12 15 18
```

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
1 2 3
4 5 6
7 8 9
```

Expected output:

```
6 15 24
12 15 18
```

Case 2 (public)**Input:**

```
0 0 1
0 1 0
1 0 0
```

Expected output:

```
1 1 1
1 1 1
```

183. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q18: Matrix Addition

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Matrix Addition. Read input from stdin and print exact output.

Input Format: r c, then matrix A (r lines), then matrix B (r lines).

Output Format: Print A+B matrix row-wise.

Constraints: Dimensions plus two matrices stacked in stdin order.

Sample Input:

```
2 2
1 2
3 4
5 6
7 8
```

Expected Output (sample):

```
6 8
10 12
```

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

```
1 1
2
7
```

Expected output:

```
9
```

Case 2 (public)**Input:**

```
2 2
1 1
1 1
2 2
2 2
```

Expected output:

```
3 3
3 3
```

Case 3 (public)**Input:**

```
1 3
1 2 3
3 2 1
```

Expected output:

```
4 4 4
```

184. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q19: Matrix Transpose

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Matrix Transpose. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Print transpose (3 lines).

Constraints: 3×3 square matrix.

Sample Input:

1 2 3

4 5 6

7 8 9

Expected Output (sample):

1 4 7

2 5 8

3 6 9

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 2 3

0 0 0

7 8 9

Expected output:

1 0 7

2 0 8

3 0 9

Case 2 (public)

Input:

1 2 3

4 5 6

7 8 9

Expected output:

1 4 7

2 5 8

3 6 9

Case 3 (public)

Input:

2 0 1

0 3 0

1 0 2

Expected output:

2 0 1

0 3 0

1 0 2

185. Phase 4 — Data Collections — Two-dimensional arrays (matrices) — Q20: Symmetric Matrix Check

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Two-dimensional arrays (matrices)'.

Problem style: Symmetric Matrix Check. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Print Yes if symmetric ($A[i][j] == A[j][i]$), else No.

Constraints: 3×3 ints.

Sample Input:

1 2 3

2 4 5

3 5 6

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1 0 0
0 1 0
0 0 1

Expected output:

Yes

Case 2 (public)

Input:

1 2 3
0 1 4
5 6 0

Expected output:

No

Case 3 (public)

Input:

1 2 3
2 4 5
3 5 6

Expected output:

Yes

186. Phase 4 — Data Collections — Strings basics and manipulation — Q21: Pattern Printing

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Pattern Printing. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print a right-angle star pattern with n lines (only the characters shown—no trailing spaces after the last character on each line).

Constraints: One height n determines how many lines of stars.

Sample Input: 3

Expected Output (sample):

*
**

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

*
**

Case 2 (public)

Input:

4

Expected output:

*
**

Case 3 (public)

Input:

5

Expected output:

*

**

187. Phase 4 — Data Collections — Strings basics and manipulation — Q22: Prime Check

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Prime Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print exactly Prime or Not (capital P / capital N as shown).

Constraints: Any integer: $n < 2$ and negatives are Not; primes are ≥ 2 with no nontrivial divisor.

Sample Input: 7

Expected Output: Prime

Judge test case count: 6

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

Prime

Case 2 (public)

Input:

9

Expected output:

Not

Case 3 (public)

Input:

17

Expected output:

Prime

Case 4 (public)

Input:

1

Expected output:

Not

Case 5 (public)

Input:

0

Expected output:

Not

Case 6 (public)

Input:

-11

Expected output:

Not

188. Phase 4 — Data Collections — Strings basics and manipulation — Q23: Print Primes In Range

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Print Primes In Range. Read input from stdin and print exact output.

Input Format: One line with two integers lo and hi separated by whitespace.

Output Format: Print all primes in [lo, hi] inclusive, space-separated, ascending.

Constraints: Two bounds lo and hi on one line (space-separated), inclusive range.

Sample Input: 5 15

Expected Output: 5 7 11 13

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2 2

Expected output:

2

Case 2 (public)

Input:

14 18

Expected output:

17

Case 3 (public)

Input:

20 30

Expected output:

23 29

189. Phase 4 — Data Collections — Strings basics and manipulation — Q24: Product Inventory Record

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Product Inventory Record. Read input from stdin and print exact output.

Input Format: n followed by n lines: product_code quantity.

Output Format: Print each product and quantity on new line.

Constraints: Count then product code strings with quantity.

Sample Input:

2

P1 10

P2 5

Expected Output (sample):

P1 10

P2 5

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

B2 10

Expected output:

B2 10

Case 2 (public)

Input:

2

C1 1

C2 2

Expected output:

C1 1

C2 2

Case 3 (public)**Input:**

3

D1 9

D2 8

D3 7

Expected output:

D1 9

D2 8

D3 7

190. Phase 4 — Data Collections — Strings basics and manipulation — Q25: Read File Content

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Read File Content. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Print the given text (simulating file content read).

Constraints: One line echoed as-if read from disk.

Sample Input: sample content

Expected Output: sample content

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

x

Expected output:

x

Case 2 (public)**Input:**

read me

Expected output:

read me

Case 3 (public)**Input:**

42

Expected output:

42

191. Phase 4 — Data Collections — Strings basics and manipulation — Q26: Reverse A String

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Reverse A String. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print reversed string.

Constraints: One word/token without spaces (matching how the checker reads input).

Sample Input: hello

Expected Output: olleh

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)**

Input:

a

Expected output:

a

Case 2 (public)

Input:

racecar

Expected output:

racecar

Case 3 (public)

Input:

coder

Expected output:

redoc

192. Phase 4 — Data Collections — Strings basics and manipulation — Q27: Check Palindrome

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Check Palindrome. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print Yes if palindrome, else No.

Constraints: One string with no gaps.

Sample Input: madam

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

Yes

Case 2 (public)

Input:

abba

Expected output:

Yes

Case 3 (public)

Input:

abc

Expected output:

No

193. Phase 4 — Data Collections — Strings basics and manipulation — Q28: Concatenate Without Strcat

Module: Phase 4 — Data Collections

Difficulty: medium

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Concatenate Without Strcat. Read input from stdin and print exact output.

Input Format: Line 1 string A; line 2 string B.

Output Format: Print A+B exactly.

Constraints: Two textual tokens sequentially lines.

Sample Input:

code

lab

Expected Output: codelab

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

x

yz

Expected output:

xyz

Case 2 (public)

Input:

tree

House

Expected output:

treeHouse

Case 3 (public)

Input:

a

b

Expected output:

ab

194. Phase 4 — Data Collections — Strings basics and manipulation — Q29: String Length Manual

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: String Length Manual. Read input from stdin and print exact output.

Input Format: Single line string (ASCII).

Output Format: Print length without using strlen.

Constraints: Non-empty string without embedded newline simplifying scanner.

Sample Input: coding

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

abcd

Expected output:

4

Case 2 (public)

Input:

race

Expected output:

4

Case 3 (public)

Input:

z

Expected output:

1

195. Phase 4 — Data Collections — Strings basics and manipulation — Q30: Count Vowels Consonants Sp

Module: Phase 4 — Data Collections

Difficulty: easy

Description: Solve a phase 4 — data collections coding task focused on 'Strings basics and manipulation'.

Problem style: Count Vowels Consonants Spaces. Read input from stdin and print exact output.

Input Format: One line: sentence.

Output Format: Print three integers: vowels (aeiou AEIOU), consonants (other letters), spaces.

Constraints: One sentence letters spaces only simplifying classification.

Sample Input: Hi there

Expected Output: 3 4 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

ABC

Expected output:

1 2 0

Case 2 (public)

Input:

a b c

Expected output:

1 2 2

Case 3 (public)

Input:

rhythm

Expected output:

0 6 0

196. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q1: Pascal Tri

Rows

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Pascal Triangle Rows. Read input from stdin and print exact output.

Input Format: One integer n = number of rows.

Output Format: Print Pascal triangle: row i has i integers, space-separated.

Constraints: `n` rows modest combinator growth.

Sample Input: 3

Expected Output (sample):

1

1 1

1 2 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

2

Expected output:

1

1 1

Case 3 (public)

Input:

5

Expected output:

1

1 1
1 2 1
1 3 3 1
1 4 6 4 1

197. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q2: Reverse A

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Reverse A String. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print reversed string.

Constraints: One word/token without spaces (matching how the checker reads input).

Sample Input: hello

Expected Output: olleh

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

a

Case 2 (public)

Input:

racecar

Expected output:

racecar

Case 3 (public)

Input:

coder

Expected output:

redoc

198. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q3: Bubble Sort

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Bubble Sort Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers.

Output Format: Print sorted ascending, space-separated (bubble sort / any stable $O(n^2)$ ok).

Constraints: n moderate sortable ints.

Sample Input:

4

4 1 3 2

Expected Output: 1 2 3 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

2 1

Expected output:

1 2

Case 2 (public)

Input:

5

5 4 3 2 1

Expected output:

1 2 3 4 5

Case 3 (public)

Input:

1

7

Expected output:

7

199. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q4: Second Largest Array

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Second Largest Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Compute M = maximum element. If some array value is smaller than M, print the largest among values strictly less than M (treat duplicate Ms as one maximum). If no value is smaller than M (all elements equal), print M.

Constraints: At least length two ints; duplicates allowed.

Sample Input:

5

3 9 9 1 4

Expected Output: 4

Judge test case count: 5

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

1 2

Expected output:

1

Case 2 (public)

Input:

4

4 4 4 4

Expected output:

4

Case 3 (public)

Input:

6

10 20 8 15 9 22

Expected output:

20

Case 4 (public)

Input:

3

100 75 90

Expected output:

90

Case 5 (public)

Input:

4

-2 -10 -10 -100

Expected output:

-10

200. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q5: String Length Manual

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: String Length Manual. Read input from stdin and print exact output.

Input Format: Single line string (ASCII).

Output Format: Print length without using strlen.

Constraints: Non-empty string without embedded newline simplifying scanner.

Sample Input: coding

Expected Output: 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

abcd

Expected output:

4

Case 2 (public)

Input:

race

Expected output:

4

Case 3 (public)

Input:

z

Expected output:

1

201. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q6: Simple Interest

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Simple Interest. Read input from stdin and print exact output.

Input Format: Three lines: principal P, rate R, time T (all integers).

Output Format: Print simple interest as integer.

Constraints: Three integers Principal Rate Time whole numbers.

Sample Input:

1000

5

2

Expected Output: 100

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

500

10

1

Expected output:

50

Case 2 (public)

Input:

1500

8

2

Expected output:

240

Case 3 (public)

Input:

9000

0

5

Expected output:

0

202. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q7: Strong Number Check

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Strong Number Check. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print Yes if sum of factorials of digits equals n, else No.

Constraints: Integer moderate digit factorial property.

Sample Input: 145

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

Yes

Case 2 (public)

Input:

2

Expected output:

Yes

Case 3 (public)

Input:

146

Expected output:

No

203. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q8: Linear Search

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Linear Search Array. Read input from stdin and print exact output.

Input Format: Line 1: n. Line 2: n integers. Line 3: target.

Output Format: Print 0-based index of first occurrence, or -1 if not found.

Constraints: Array listing plus target finder.

Sample Input:

```
5
1 4 7 9 2
7
```

Expected Output: 2

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
4
8 8 8 8
8
```

Expected output:

```
0
```

Case 2 (public)

Input:

```
2
1 2
3
```

Expected output:

```
-1
```

Case 3 (public)

Input:

```
1
42
42
```

Expected output:

```
0
```

204. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q9: Count Vowels Consonants Spaces

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Count Vowels Consonants Spaces. Read input from stdin and print exact output.

Input Format: One line: sentence.

Output Format: Print three integers: vowels (aeiou AEIOU), consonants (other letters), spaces.

Constraints: One sentence letters spaces only simplifying classification.

Sample Input: Hi there

Expected Output: 3 4 1

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

```
ABC
```

Expected output:

```
1 2 0
```

Case 2 (public)

Input:

a b c

Expected output:

1 2 2

Case 3 (public)

Input:

rhythm

Expected output:

0 6 0

205. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q10: Palindromer

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Palindrome Number. Read input from stdin and print exact output.

Input Format: One integer n (no leading zeros except single 0).

Output Format: Print Yes if decimal digits read same forward/backward, else No.

Constraints: Integer decimal digit symmetry checking.

Sample Input: 121

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

0

Expected output:

Yes

Case 2 (public)

Input:

1221

Expected output:

Yes

Case 3 (public)

Input:

10

Expected output:

No

206. Phase 5 — Problem Practice — Mixed array, matrix, string, and number-theory drills — Q11: Find Sumay

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Mixed array, matrix, string, and number-theory drills'.

Problem style: Find Sum Of Array. Read input from stdin and print exact output.

Input Format: First line n, second line n integers.

Output Format: Print sum of elements.

Constraints: First n, then exactly n integers on the next input line.

Sample Input:

5

1 2 3 4 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1
10

Expected output:

10

Case 2 (public)

Input:

4
0 0 0 0

Expected output:

0

Case 3 (public)

Input:

3
7 8 9

Expected output:

24

207. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q12: Student Record

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Student Record System. Read input from stdin and print exact output.

Input Format: n followed by n lines: name age.

Output Format: Print all student records line by line.

Constraints: Count then lines formatted name + integer age.

Sample Input:

2
Alice 20
Bob 22

Expected Output (sample):

Alice 20
Bob 22

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2
Tom 20
Sam 19

Expected output:

Tom 20
Sam 19

Case 2 (public)

Input:

1
Eve 25

Expected output:

Eve 25

Case 3 (public)

Input:

3

- A 1
- B 2
- C 3

Expected output:

- A 1
- B 2
- C 3

208. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q13: Sum Using Recursion

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Sum Using Recursion. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print sum of numbers from 1 to n using recursion.

Constraints: One positive integer n.

Sample Input: 5

Expected Output: 15

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

3

Case 2 (public)

Input:

10

Expected output:

55

Case 3 (public)

Input:

7

Expected output:

28

209. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q14: Swap Using Pointers

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Swap Using Pointers. Read input from stdin and print exact output.

Input Format: Two integers a and b.

Output Format: Print swapped numbers using pointer logic.

Constraints: Two integers as usual swap problem.

Sample Input: 4 9

Expected Output: 9 4

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

-2 5

Expected output:

5 -2

Case 2 (public)**Input:**

8 3

Expected output:

3 8

Case 3 (public)**Input:**

0 7

Expected output:

7 0

210. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q15: Symmetric Mat

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Symmetric Matrix Check. Read input from stdin and print exact output.

Input Format: Three lines, each three integers.

Output Format: Print Yes if symmetric ($A[i][j] == A[j][i]$), else No.Constraints: 3×3 ints.**Sample Input:**

1 2 3

2 4 5

3 5 6

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):**Case 1 (public)****Input:**

1 0 0

0 1 0

0 0 1

Expected output:

Yes

Case 2 (public)**Input:**

1 2 3

0 1 4

5 6 0

Expected output:

No

Case 3 (public)**Input:**

1 2 3

2 4 5

3 5 6

Expected output:

Yes

211. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q16: Write Data To F

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Write Data To File. Read input from stdin and print exact output.

Input Format: A single line of text.

Output Format: Echo the same text (simulating file write/read).

Constraints: Single textual line echoed (simulates file write).

Sample Input: hello

Expected Output: hello

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

abc

Expected output:

abc

Case 2 (public)

Input:

123

Expected output:

123

Case 3 (public)

Input:

file data

Expected output:

file data

212. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q17: Matrix Print 3X3

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Matrix Print 3X3. Read input from stdin and print exact output.

Input Format: Three lines, each with three integers.

Output Format: Print matrix row-wise with spaces between numbers per row.

Constraints: Exactly nine ints row major three lines typed.

Sample Input:

1 2 3

4 5 6

7 8 9

Expected Output (sample):

1 2 3

4 5 6

7 8 9

Judge test case count: 2

All test cases (for evaluation / marking):

Case 1 (public)

Input:

9 8 7

6 5 4

3 2 1

Expected output:

9 8 7

6 5 4

3 2 1

Case 2 (public)

Input:

1 0 0

0 1 0

0 0 1

Expected output:

1 0 0

0 1 0

0 0 1

213. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q18: Concatenate Without Strcat

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Concatenate Without Strcat. Read input from stdin and print exact output.

Input Format: Line 1 string A; line 2 string B.

Output Format: Print A+B exactly.

Constraints: Two textual tokens sequentially lines.

Sample Input:

code

lab

Expected Output: codelab

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

x

yz

Expected output:

xyz

Case 2 (public)

Input:

tree

House

Expected output:

treeHouse

Case 3 (public)

Input:

a

b

Expected output:

ab

214. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q19: Fibonacci Series

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Fibonacci Series. Read input from stdin and print exact output.

Input Format: One integer n (number of terms).

Output Format: Print first n Fibonacci numbers separated by spaces.

Constraints: Count of terms to emit.

Sample Input: 7

Expected Output: 0 1 1 2 3 5 8

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

0 1

Case 2 (public)

Input:

3

Expected output:

0 1 1

Case 3 (public)

Input:

6

Expected output:

0 1 1 2 3 5

215. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q20: Check Palindromes

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Check Palindrome. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print Yes if palindrome, else No.

Constraints: One string with no gaps.

Sample Input: madam

Expected Output: Yes

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

Yes

Case 2 (public)

Input:

abba

Expected output:

Yes

Case 3 (public)

Input:

abc

Expected output:

No

216. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q21: Reverse A String

Module: Phase 5 — Problem Practice

Difficulty: easy

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Reverse A String. Read input from stdin and print exact output.

Input Format: One string (no spaces).

Output Format: Print reversed string.

Constraints: One word/token without spaces (matching how the checker reads input).

Sample Input: hello

Expected Output: olleh

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

a

Expected output:

a

Case 2 (public)

Input:

racecar

Expected output:

racecar

Case 3 (public)

Input:

coder

Expected output:

redoc

217. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q22: Pattern Printing

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Pattern Printing. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print a right-angle star pattern with n lines (only the characters shown—no trailing spaces after the last character on each line).

Constraints: One height n determines how many lines of stars.

Sample Input: 3

Expected Output (sample):

*

**

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

2

Expected output:

*

**

Case 2 (public)

Input:

4

Expected output:

*

**

Case 3 (public)

Input:

5

Expected output:

*

**

218. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q23: Factorial Of Number

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Factorial Of Number. Read input from stdin and print exact output.

Input Format: One integer n.

Output Format: Print n!.

Constraints: Small non-negative integer n (factorial grows fast).

Sample Input: 5

Expected Output: 120

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

6

Expected output:

720

Case 3 (public)

Input:

3

Expected output:

6

219. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q24: Equilateral Star Pattern

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Equilateral Star Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows in the centered star triangle).

Output Format: For each row r from 0 to n-1 (top to bottom): print exactly (n-1-r) spaces, then (r+1) stars with a single space between consecutive stars, and no space after the last star on that line. This centers each row so column alignment matches the sample for every test case.

Constraints: Same spacing rule for every row: row r uses (n-1-r) spaces then r+1 stars with single spaces between stars; no trailing spaces.

Sample Input: 4

Expected Output (sample):

```
*  
* *  
* * *  
* * * *
```

Judge test case count: 4

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

*

Case 2 (public)

Input:

2

Expected output:

*

* *

Case 3 (public)

Input:

3

Expected output:

*

* *

* * *

Case 4 (public)

Input:

4

Expected output:

*

* *

* * *

* * * *

220. Phase 5 — Problem Practice — Patterns, sorts, palindromes, gcd/lcm bundles — Q25: Floyd Triangle

Module: Phase 5 — Problem Practice

Difficulty: medium

Description: Solve a phase 5 — problem practice coding task focused on 'Patterns, sorts, palindromes, gcd/lcm bundles'.

Problem style: Floyd Triangle Pattern. Read input from stdin and print exact output.

Input Format: One integer n (number of rows).

Output Format: Floyd triangle: row i has i consecutive integers, numbers separated by a single space; rows separated by newline. Each line ends immediately after the last digit—no trailing spaces.

Constraints: Height rows grows consecutive integers pattern.

Sample Input: 3

Expected Output (sample):

1

2 3

4 5 6

Judge test case count: 3

All test cases (for evaluation / marking):

Case 1 (public)

Input:

1

Expected output:

1

Case 2 (public)

Input:

4

Expected output:

1

2 3

4 5 6

7 8 9 10

Case 3 (public)

Input:

3

Expected output:

1

2 3

4 5 6